DECLARAÇÃO DO(A) ORIENTADOR(A)

(Concordância com a Composição da Banca Examinadora)

UNIVERSIDADE FEDERAL DO CEARÁ
Pró-Reitoria de Pesquisa e Pós-Graduação
Programa de Pós-Graduação em Ciências da Computação

DECLARAÇÃO DE CONCORDÂNCIA DO(A) ORIENTADOR(A) COM A COMPOSIÇÃO DA BANCA EXAMINADORA

Eu, César Lincoln Cavalcante Mattos, professor(a) vinculado(a) ao Programa de Pós-Graduação em Ciências da Computação da Universidade Federal do Ceará, matrícula SIAPE nº 1407874, declaro, para os devidos fins, que concordo com a composição da banca examinadora sugerida para a defesa de dissertação de mestrado do(a) discente Hugo Ramos Soares, matrícula nº 564305, a ser realizada no dia vinte e oito de agosto de 2026.

A banca será composta pelos seguintes membros:

Presidente (Orientador(a)):
Prof. Dr. César Lincoln Cavalcante Mattos – Universidade Federal do Ceará (UFC)

Membros Titulares:
Prof. Dr. João Paulo Pordeus Gomes – Universidade Federal do Ceará (UFC)
Prof. Dr. José Antônio Fernandes de Macêdo – Universidade Federal do Ceará (UFC)
Prof. Dr. Diego Parente Paiva Mesquita – Fundação Getúlio Vargas (FGV)

Declaro, ainda, que a composição acima atende aos critérios estabelecidos pelo Programa de Pós-Graduação em Ciências da Computação, bem como às normas da Universidade Federal do Ceará e da CAPES para a realização de defesas acadêmicas no âmbito da pós-graduação stricto sensu.

Por ser verdade, firmo a presente declaração.

Fortaleza, 13 de Agosto de 2026.

_____

Prof. Dr. César Lincoln Cavalcante Mattos
Orientador(a)
Programa de Pós-Graduação em Ciências da Computação
Universidade Federal do Ceará

DECLARAÇÃO SOBRE O USO OU NÃO USO DE INTELIGÊNCIA ARTIFICIAL

Eu, Hugo Ramos Soares, responsável pela obra HoTHP: Hyperbolic Rotary Position Embedding-based Transformer Hawkes Process for Length Extrapolation, declaramos que as informações prestadas refletem de forma verdadeira e completa o uso ou não uso de ferramentas de Inteligência Artificial nesta produção, em conformidade com a Resolução de Integridade da UFC e a PORTARIA Nº 39/PRPPG/UFC, DE 01 DE OUTUBRO DE 2025.

SELECIONE UMA DAS OPÇÕES:

☐ **NÃO HOUVE USO DE FERRAMENTAS DE INTELIGÊNCIA ARTIFICIAL**

Declaro que não utilizei, em nenhuma etapa do desenvolvimento do presente trabalho, ferramentas de Inteligência Artificial, incluindo mas não se limitando a recursos de geração de texto, imagem, código, resumos, traduções ou análises. Assumo integral responsabilidade pelo conteúdo, conforme os princípios de integridade acadêmica e a legislação vigente.

(Se marcar esta opção, não é necessário preencher os campos 1 a 5 abaixo.)

☑ **HOUVE USO DE FERRAMENTAS DE INTELIGÊNCIA ARTIFICIAL**

(Preencher os campos obrigatórios abaixo.)

1. Ferramenta(s) e versão(ões) predominante(s): Claude Opus 4.6, 4.7 e 4.8
2. Período de uso: Junho/2025 a Junho/2026
3. Finalidade(s) (marcar todas que se aplicam):
   - ☑ Exploração inicial de ideias
   - ☐ Busca/triagem de literatura
   - ☐ Leitura assistida/Resumos (com conferência humana)
   - ☐ Revisão linguística/Referências
   - ☑ Apoio reprodutível à análise/apresentação (restrito à geração automatizada e reprodutível de tabelas, gráficos, figuras ou visualizações a partir de dados previamente analisados e interpretados pelos autores, sem substituição da análise científica humana)
   - ☑ Programação (sugestão/depuração/documentação)
   - ☐ Transcrição (com anonimização): conversão literal, mediante autorização, de áudios ou vídeos em texto de entrevistas, aulas, reuniões ou palestras, com obrigatória revisão humana, anonimização e respeito aos direitos autorais, vedada a identificação de voz ou outros dados biométricos.
   - ☑ Tradução técnica (com revisão humana)

4. Descrição sintética do uso / prompts-tipo: Usei o Claude para discutir conceitos de arquiteturas de Transformer Hawkes Processes (THP, RoTHP, kernel hiperbólico vs. trigonométrico), gerar scripts de tabelas, gráficos e figuras a partir de resultados que eu já tinha obtido e interpretado, sugerir, debugar e documentar código e traduzir o abstract e trechos técnicos para o inglês. Prompts como "explique a diferença entre kernel trigonométrico e hiperbólico", "gere um gráfico comparando essas métricas", "traduza este parágrafo técnico".

5. Validação humana (checagens, testes, leitura crítica): Conferi as explicações conceituais com a literatura, executei e testei todo o código sugerido, reproduzindo os resultados, verifiquei figuras e tabelas e revisei as traduções linha a linha. A análise, interpretação e conclusões são minhas e assumo responsabilidade integral pelo conteúdo.

Declarações éticas (obrigatórias caso tenha havido uso de IA):

Todas as opções abaixo devem ser obrigatoriamente assinaladas para que o trabalho possa ser submetido à defesa/apresentação. O não preenchimento completo impedirá a continuidade do processo.

- ☑ Não houve geração de conteúdo original, ideias, interpretações ou análises pela IA;
- ☑ Não enviei dados inéditos ou sensíveis à serviços que utilizam conteúdos para treinamento de modelos, exceto em plataformas institucionais ou com garantias contratuais de confidencialidade e não retenção, assegurando conformidade com a LGPD (Lei nº 13.709/2018) e demais normas de proteção de dados;
- ☑ Respeitei direitos autorais, licenças, confidencialidade e políticas editoriais;
- ☑ Em transcrições, apliquei anonimização e não realizei identificação por voz/biometria;
- ☑ Assumo responsabilidade integral e exclusiva pelo conteúdo final desta obra.

Data: 13 de agosto de 2026
Nome do Autor: Hugo Ramos Soares

Assinatura: _____

NOME DO DOCUMENTO

Hugo Ramos - Dissertação de Mestrado

AUTOR

Hugo Ramos

NÚMERO DE PALAVRAS

23837 Words

NÚMERO DE CARACTERES

116789 Characters

NÚMERO DE PÁGINAS

75 Pages

TAMANHO DO ARQUIVO

2.6MB

DATA DE ENVIO

Aug 17, 2026 2:02 PM GMT-3

DATA DO RELATÓRIO

Aug 17, 2026 2:03 PM GMT-3

16% geral de similaridade

O total combinado de todas as correspondências, incluindo fontes sobrepostas, para cada banco de dados.

- 12% Banco de dados da Internet
- Banco de dados do Crossref
- 10% Banco de dados de trabalhos enviados
- 10% Banco de dados de publicações
- Banco de dados de conteúdo publicado no Crossref

UNIVERSIDADE FEDERAL DO CEARÁ
CENTRO DE CIÊNCIAS
DEPARTAMENTO DE COMPUTAÇÃO
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO
MESTRADO ACADÊMICO EM LÓGICA E INTELIGÊNCIA ARTIFICIAL

HUGO RAMOS SOARES

HOTHP: HYPERBOLIC ROTARY POSITION EMBEDDING-BASED TRANSFORMER HAWKES PROCESS FOR LENGTH EXTRAPOLATION

FORTALEZA
2026

HUGO RAMOS SOARES

HOTHP: HYPERBOLIC ROTARY POSITION EMBEDDING-BASED TRANSFORMER HAWKES PROCESS FOR LENGTH EXTRAPOLATION

Dissertação apresentada ao Curso de Mestrado Acadêmico em Lógica e Inteligência Artificial do Programa de Pós-Graduação em Ciência da Computação do Centro de Ciências da Universidade Federal do Ceará, como requisito parcial à obtenção do título de mestre em Ciência da Computação. Área de Concentração: Ciência da Computação

Orientador: Prof. Dr. César Lincoln Cavalcante Mattos

Coorientador: Prof. Dr. João Paulo Pordeus Gomes

FORTALEZA

2026

HUGO RAMOS SOARES

HOTHP: HYPERBOLIC ROTARY POSITION EMBEDDING-BASED TRANSFORMER HAWKES PROCESS FOR LENGTH EXTRAPOLATION

Dissertação apresentada ao Curso de Mestrado Acadêmico em Lógica e Inteligência Artificial do Programa de Pós-Graduação em Ciência da Computação do Centro de Ciências da Universidade Federal do Ceará, como requisito parcial à obtenção do título de mestre em Ciência da Computação. Área de Concentração: Ciência da Computação

Aprovada em:

BANCA EXAMINADORA

Prof. Dr. César Lincoln Cavalcante Mattos (Orientador)
Universidade Federal do Ceará (UFC)

Prof. Dr. João Paulo Pordeus Gomes (Coorientador)
Universidade Federal do Ceará (UFC)

Prof. Dr. José Antônio Fernandes de Macêdo
Universidade Federal do Ceará (UFC)

Prof. Dr. Diego Parente Paiva Mesquita
Fundação Getúlio Vargas (FGV)

Dedico este trabalho à minha mãe, que sempre acreditou em mim. À minha mulher, que esteve ao meu lado em cada passo. E aos meus filhos, Raul e Lucas, razão da minha vida.

AGRADECIMENTOS

Ao meu orientador, Prof. Dr. César Lincoln Cavalcante Mattos, pela orientação sempre atenta, pelo acompanhamento durante toda a pesquisa e, em especial, pela paciência nas muitas vezes em que mudei de ideia ao longo do caminho.

Ao meu coorientador, Prof. Dr. João Paulo Pordeus Gomes, que foi meu primeiro contato na UFC e se mostrou dedicado desde o começo, a ponto de me explicar redes convolucionais em uma folha de papel logo na primeira vez que conversamos pessoalmente.

Aos professores que compuseram a banca examinadora, Prof. Dr. José Antônio Fernandes de Macêdo, de quem também tive a satisfação de ser aluno e que tornou a ciência de dados mais interessante com tantas aplicações reais, e Prof. Dr. Diego Parente Paiva Mesquita, pela disponibilidade e pelas contribuições que enriqueceram este trabalho.

Ao Prof. Dr. Ronaldo Menezes, professor visitante no programa, pelo entusiasmo com que aproximou a turma de um tema novo e pela perspectiva diferente que trouxe de sua experiência no exterior.

Ao amigo e colega Manolidis Efstratios Júnior, que entrou no mestrado junto comigo e com quem dividi quase todas as disciplinas e as reuniões de orientação. Fazer esse percurso lado a lado tornou tudo mais leve.

Ao MDCC (Mestrado e Doutorado em Ciência da Computação) da Universidade Federal do Ceará, pelo ensino público e de qualidade e pela estrutura que tornou este mestrado possível.

Ao Doutorando em Engenharia Elétrica, Ednardo Moreira Rodrigues, e seu assistente, Alan Batista de Oliveira, aluno de graduação em Engenharia Elétrica, pela adequação do *template* utilizado neste trabalho para que o mesmo ficasse de acordo com as normas da biblioteca da Universidade Federal do Ceará (UFC).

“Não é porque as coisas são difíceis que não ousamos; é porque não ousamos que elas são difíceis.”

(Sêneca)

RESUMO

Processos Pontuais Temporais (*Temporal Point Process* (TPP)) modelam fluxos de eventos em tempo contínuo, e os TPPs baseados em Transformers se tornaram uma ferramenta forte para essa tarefa. Na prática, muitas vezes precisamos treinar em sequências curtas e depois rodar o modelo em sequências bem mais longas, e é exatamente aí que esses modelos falham: sua acurácia colapsa quando as sequências de teste são mais longas do que as vistas no treinamento. Neste trabalho, propomos o HoTHP (Hyperbolic Rotary Position Embedding-based Transformer Hawkes Process), um modelo Transformer para processos de Hawkes que incorpora um viés de recência diretamente no kernel de atenção. Em vez do kernel trigonométrico oscilante do RoTHP (Rotary Position Embedding-based Transformer Hawkes Process), o HoTHP usa um kernel hiperbólico que decai de forma suave e monotônica com o tempo entre os eventos, de modo que o modelo se mantém previsível mesmo em distâncias que nunca viu no treinamento. O decaimento é definido por um único parâmetro aprendido e é monotônico por construção. Como esse viés atua na atenção e não na função de intensidade, o HoTHP ainda modela tanto processos auto-excitatórios quanto auto-inibitórios. Avaliamos o modelo sob o protocolo train-short, test-long, com sequências de teste até $5\times$ mais longas que o treinamento, em dados sintéticos de Hawkes e em quatro datasets reais (Amazon, Stackoverflow, Taxi e Retweet). Nos dados sintéticos, a negativa do log da verossimilhança (*Negative Log-Likelihood* (NLL)) do HoTHP se mantém estável à medida que as sequências crescem, enquanto o RoTHP degrada. Nos dados reais, o HoTHP obtém a melhor NLL em extrapolação entre os modelos Transformer, e no Retweet a diferença é drástica ($-0{,}651$ contra $47{,}6$ do RoTHP, após normalização temporal). Seu ganho mais claro está na acurácia de predição de tipo: o HoTHP é o modelo mais acurado no Amazon e no Stackoverflow, à frente até do forte baseline recorrente Neural Hawkes Process (NHP). Seu custo adicional de treinamento ($1{,}05\times$ a $2{,}24\times$) é pago uma única vez e garante inferência estável em sequências bem mais longas.

Palavras-chave: Processos Pontuais Temporais. Processo de Hawkes. Transformers. Extrapolação de comprimento.

ABSTRACT

Temporal Point Processes (TPP) model streams of events in continuous time, and Transformer-based TPPs have become a strong tool for this task. In practice, we often need to train on short sequences and then run the model on much longer ones, and this is exactly where these models fail: their accuracy collapses when the test sequences are longer than those seen in training. This work proposes HoTHP (Hyperbolic Rotary Position Embedding-based Transformer Hawkes Process), a Transformer model for Hawkes processes that builds a recency bias directly into the attention kernel. Instead of the oscillating trigonometric kernel of RoTHP (Rotary Position Embedding-based Transformer Hawkes Process), HoTHP uses a hyperbolic kernel that decays smoothly and monotonically with the time between events, so the model stays predictable even at distances it never saw in training. The decay is set by a single learned parameter and is monotonic by construction. Because this bias acts on the attention and not on the intensity function, HoTHP still models both self-exciting and self-inhibiting processes. We evaluate it under a train-short, test-long protocol, with test sequences up to 5× longer than training, on synthetic Hawkes data and on four real datasets (Amazon, Stackoverflow, Taxi, and Retweet). On synthetic data, the negative log-likelihood (NLL) of HoTHP stays stable as the sequences grow, while RoTHP degrades. On real data, HoTHP has the best extrapolation NLL among the Transformer models, and on Retweet the gap is drastic (-0.651 against 47.6 for RoTHP, after temporal normalization). Its clearest gain is in type-prediction accuracy: HoTHP is the most accurate model on Amazon and Stackoverflow, ahead even of the strong recurrent Neural Hawkes Process (NHP) baseline. Its extra training cost (1.05× to 2.24×) is paid only once and buys stable inference on much longer sequences.

Keywords: Temporal Point Process. Hawkes Process. Transformers. Length extrapolation.

LISTA DE FIGURAS

LIST OF TABLES

LISTA DE ABREVIATURAS E SIGLAS

ALiBi	*Attention with Linear Biases*
HoPE	*Hyperbolic Rotary Positional Encoding for Stable Long-Range Dependency Modeling in Large Language Models*
HoTHP	*Hyperbolic Rotary Position Embedding-based Transformer Hawkes Process*
LSTM	*Long Short-Term Memory*
NHP	*Neural Hawkes Process*
NLL	*Negative Log-Likelihood*
RMSE	*Root Mean Squared Error*
RoTHP	*Rotary Position Embedding-based Transformer Hawkes Process*
SAHP	*Self-Attentive Hawkes Process*
THP	*Transformer Hawkes Process*
TPP	*Temporal Point Process*

LIST OF SYMBOLS

b_k	Baseline (inertia) term of the intensity of type k
$\mathbf{B}_j$	Hyperbolic rotation block of the HoTHP for the pair of dimensions j
d	Dimension of each attention head
$\mathscr{H}_t$	History of the events up to time t
$\mathbf{h}(t_j)$	Hidden state produced by the encoder for event j
k	Event type (mark)
K	Number of event types
L	Training sequence length
N	Number of events in a sequence
$\mathbf{q}, \mathbf{k}, \mathbf{v}$	Query, key, and value vectors of the attention
$\mathbf{R}_j$	Trigonometric rotation block of the RoTHP for the pair of dimensions j
t_i, t_j	Timestamps of events i and j
$\mathbf{w}_k$	Weight vector of the intensity of type k
α	Excitation parameter of the Hawkes process (size of the jump)
β	Decay rate of the Hawkes process
γ_k	Coefficient of the elapsed-time term in the intensity of type k (α_k in the original THP)
Δt	Temporal difference between two events, $t_i - t_j$
δ	Decay rate of the continuous-time LSTM (NHP)
ε	Small positive constant (numerical safety margin)
θ_j	Frequencies of the positional kernel (RoPE and HoPE)
θ'	Learnable decay parameter of the hyperbolic kernel
$\lambda(t \mid \mathscr{H}_t)$	Conditional intensity function
λ_k	Conditional intensity of type k
Λ	Compensator, the integrated intensity $\int \lambda(t)\, dt$
μ	Baseline intensity of the Hawkes process
ρ	Extrapolation factor (test length divided by training length)

σ	Sigmoid function
$\varphi(\Delta t)$	Hawkes kernel, $\alpha e^{-\beta \Delta t}$

CONTENTS

1 INTRODUCTION

Many real-world phenomena can be modeled as sequences of events that happen at irregular instants over time, such as financial transactions, earthquakes, interactions in social networks, neuron firings, and so on. These events are asynchronous and usually influence one another, which makes the study of such phenomena both complex and necessary (ZHOU *et al.*, 2025). Temporal point processes (TPPs) are the mathematical framework for modeling these sequences. The central goal is to estimate the conditional intensity function $\lambda^*(t)$, which describes the instantaneous rate of event occurrence given the history.

Among the classical models, the Hawkes process (HAWKES, 1971) stands out for capturing self-excitation, that is, when the occurrence of an event temporarily increases the probability of future events, with the influence decaying exponentially over time. More recent neural models, such as the Transformer Hawkes Process (*Transformer Hawkes Process* (THP)) (ZUO *et al.*, 2021), replace the parametric kernel with an attention mechanism that learns the relationship between events directly from the data. The *Rotary Position Embedding-based Transformer Hawkes Process* (RoTHP) (DAI *et al.*, 2026) moves further in this direction by using rotary positional embeddings (RoPE) (SU *et al.*, 2023) to encode relative temporal distances in the attention mechanism.

An open problem in these models is length extrapolation. When they are trained on short sequences and tested on longer ones, performance tends to degrade. In the THP, this happens because the absolute temporal encoding produces representations that fall outside the training distribution for new positions. In the RoTHP, the trigonometric kernel (based on sine and cosine) is oscillatory, and for temporal distances not seen during training the attention score can take arbitrary values, with no guarantee of decay. This limitation matters in practice: in many real-world scenarios, collecting long sequences is expensive or impractical, so it is desirable to train on shorter sequences and run the model on arbitrarily longer ones.

This work proposes the *Hyperbolic Rotary Position Embedding-based Transformer Hawkes Process* (HoTHP), which replaces the trigonometric kernel of the RoTHP with a hyperbolic kernel that has exponential decay, inspired by the *Hyperbolic Rotary Positional Encoding for Stable Long-Range Dependency Modeling in Large Language Models* (HoPE) (DAI *et al.*, 2025). The idea is to embed a recency bias into the attention mechanism, so that recent events receive larger weights and distant events receive weights that tend monotonically to zero. This bias is structural (it does not depend on the training data) and holds for arbitrarily large temporal

distances, which makes extrapolation possible.

1.1 Objectives

The general objective of this work is to investigate whether replacing the oscillatory kernel with a kernel that has exponential decay in the attention mechanism improves the length extrapolation ability of neural models for temporal point processes.

To reach this, the following specific objectives are defined:

- Propose and implement the HoTHP, adapting the HoPE (DAI *et al.*, 2025) from discrete positions to continuous instants.
- Evaluate the model with the *train short, test long* technique on synthetic data, where the generating process is known, and on real data from varied domains.
- Analyze under which conditions the recency bias is beneficial and under which it is neutral or harmful, through the visualization of the learned intensities, accuracy, *Root Mean Squared Error* (RMSE), and ablations on the temporal scale of the data.

1.2 Contributions

This work proposes the HoTHP, which introduces an exponential recency bias directly into the temporal attention kernel. The kernel uses a decay parameter θ' that is learnable and structurally constrained to guarantee monotonic decay. The experimental analysis shows that this bias acts on the construction of the hidden state (through attention) and not on the intensity function, which allows the model to learn both self-exciting and self-inhibiting processes, as long as recent events are more informative than distant ones. The evaluation on synthetic data and four real datasets identifies the conditions that determine when the bias is beneficial and when it is not (processes without memory or with an extreme temporal scale).

1.3 Text organization

The remainder of this dissertation is organized as follows. Chapter 2 presents the theoretical background on point processes, Hawkes processes, neural Hawkes processes (NHP), and the Transformer architecture with its attention mechanism. Chapter 3 reviews the related work: the Transformer-based approaches for point processes (THP and RoTHP) and the length extrapolation and inductive bias techniques (RoPE, ALiBi) that underlie the HoTHP. Chapter 4

describes the HoTHP architecture, the formulation of the hyperbolic kernel, and the experimental setup. Chapter 5 presents the results on synthetic and real data, the learned intensities, and the temporal normalization ablation. Chapter 6 concludes the work and proposes future directions.

2 THEORETICAL BACKGROUND

Understanding the behavior of event sequences over time and how events influence one another requires the use of complex mathematical tools. This chapter presents the theoretical background needed to support the proposed model. We start with the Poisson distribution and the modeling of point processes, which are needed to understand how events happen in continuous time in an irregular way. Next, we look at the Hawkes process, which shows how events can affect the probability of future events. After that, we present neural Hawkes processes, which replace the parametric kernel with neural networks, culminating in the *Neural Hawkes Process* (NHP), and we review the Transformer architecture and its attention mechanism. The Transformer-based Hawkes process approaches (THP and RoTHP) and the length extrapolation techniques, which are the direct basis of the proposed model, are covered in Chapter 3.

2.1 Poisson distribution

Before dealing with the Poisson process, we first recall the Poisson distribution, from which it takes its name. The Poisson distribution is a discrete probability distribution that models the number of events that occur in a fixed interval of time or space, under the assumptions that the events are independent and occur at a constant average rate λ. The probability of observing exactly k events is given by:

$$\Pr(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \ldots, \tag{2.1}$$

where $\lambda > 0$ is the average number of events expected in the interval. Both the mean and the variance of this distribution are equal to λ, so that a higher rate implies both more expected events and greater variability around that expectation.

The Poisson distribution can also be obtained as an approximation to the binomial distribution. If X counts the number of successes in n independent trials, each with success probability p, and we write $\lambda = np$, then when n is large and p is small enough that np is of moderate size, the binomial probabilities are well approximated by the expression in Equation (2.1) (ROSS, 2019). This justifies the use of the Poisson distribution to count independent occurrences when each one is individually unlikely but a large number of opportunities is available.

The same distribution underlies the Poisson process presented in the next section, which generates events over time so that the count in any interval is Poisson distributed. There,

Equation (2.4) reappears with $\Lambda = \lambda T$ in place of λ, so that the expected number of events grows proportionally to the length T of the observed interval.

2.2 Poisson processes

The Poisson process is a mathematical object that consists of random points in a mathematical space with the important feature of no dependence between the points. This model is widely used in areas such as queueing theory, astronomy, biology, and survival analysis. The name comes from the fact that the points, or events, occur following a Poisson distribution.

The Poisson process is the most common form of point process. Daley e Vere-Jones (2003) define a stationary Poisson process by the following equation, where we use $N(a_i, b_i]$ to represent the number of events of the process.

$$\Pr\{N((a_i,b_i]) = n_i,\ i = 1,\ldots,k\} = \prod_{i=1}^{k} \frac{[\lambda(b_i - a_i)]^{n_i}}{n_i!} e^{-\lambda(b_i - a_i)}. \tag{2.2}$$

This definition covers three important aspects: (i) the number of points in each finite interval $(a_i, b_i]$ has a Poisson distribution; (ii) the numbers of points in disjoint intervals are independent random variables; and (iii) the distributions are stationary, that is, they depend only on the lengths $b_i - a_i$ of the intervals. The constant λ is the average rate or the average density of the events of the process. In the following subsections we will see that λ is also the intensity with which the events occur.

Considering the special case of a single interval $k = 1$, we have $(a_i, b_i] = (0, T]$. Then, the interval size is $b_i - a_i = T$. Substituting into Equation (2.2), we have:

$$\Pr\{N((0,T]) = n_1\} = \frac{[\lambda T]^{n_1}}{n_1!} e^{-\lambda T}. \tag{2.3}$$

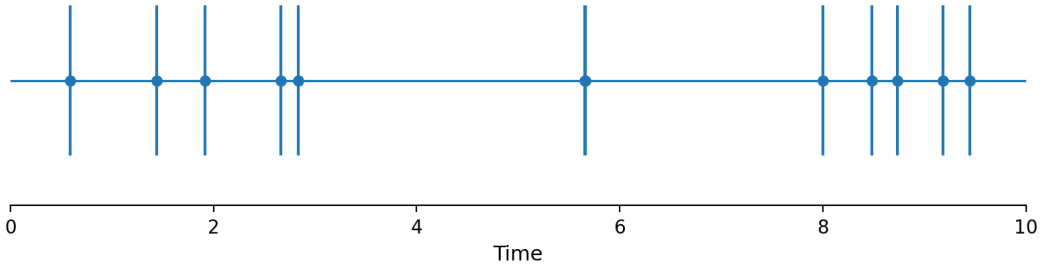
In Equation (2.3), replacing $(0, T]$ by N and λT by Λ, we obtain the classical form of the Poisson distribution associated with the Poisson process:

$$\Pr(N = n) = \frac{\Lambda^n}{n!} e^{-\Lambda}. \tag{2.4}$$

Equation (2.4) makes explicit the relationship between the Poisson distribution and the Poisson process: the number of events observed in any interval of size T is Poisson with mean $\Lambda = \lambda T$. In other words, the Poisson process is a way of generating event times whose count satisfies the Poisson distribution and that are independent and stationary. Poisson processes are the most fundamental form of temporal point process, which we will discuss in the next section.

Figure 1 illustrates a homogeneous Poisson process in the interval $[0, T]$. The horizontal axis represents time and each vertical mark represents the occurrence of an event generated by the process. Although the events look irregular, they were generated so that the waiting time between one event and another is exponential $1/\lambda$ and independent, where λ is constant and therefore the expected number of events in a given interval T is defined by λT. This illustrates the intuition behind the Poisson process, which is the random occurrence of events, independent and following a constant rate λ.

Figure 1 – Poisson process.



2.3 Temporal point processes

Temporal point processes (TPP) are probabilistic models designed to describe sequences of discrete events that happen in continuous time (ZUO *et al.*, 2021). Unlike discrete models, which assume that events happen at known time intervals, TPPs make it possible to handle events that happen at irregular, arbitrary times. This makes them particularly useful in domains where the moment of the event occurrence is as important as the event itself.

TPPs are formally defined by a conditional intensity function $\lambda(t \mid \mathcal{H}_t)$, where $\mathcal{H}_t$ is the history of events up to time t. The intensity is the instantaneous rate at which an event is expected to occur at a time t, given the past events.

One variation of TPPs is the existence of *marks*, that is, event types, where the sequence is now described as $X = \{(t_1, m_1), \ldots, (t_N, m_N)\}$, where N represents the number of events and each m_i is the mark (type) associated with event i.

The term *mark* is not used in the same way by every author. Part of the literature treats the mark as the event type, that is, a category from a finite set. This is the view of Du *et al.* (2016), who represent the mark as a one-hot vector and predict it as one class among K. Another part of the literature treats the mark in a broader sense, as any extra information attached to an

event, which may be continuous. Rasmussen (2018), for example, describes the mark as the additional information associated with an event, such as the magnitude of an earthquake, and lets the mark space be a subset of $\mathbb{R}$ or of $\mathbb{N}$. In this work we adopt the first convention: the mark is always the event type, a categorical value, which is the discrete case ($\mathbb{N}$) of the broader definition. We therefore use the words *mark* and *type* interchangeably throughout the text.

The distribution of a TPP with K marks can be characterized by K conditional intensity functions $\lambda_k(t \mid \mathcal{H}_t)$ (one for each mark k) and is defined as

$$\lambda_k(t \mid \mathcal{H}_t) = \lim_{\Delta t \to 0} \frac{\Pr(\text{event of type } k \text{ in } [t, t+\Delta t) \mid \mathcal{H}_t)}{\Delta t}. \tag{2.5}$$

Note that the definition above can also be applied to TPPs without marks, simply by setting $K = 1$ (SHCHUR *et al.*, 2021).

2.3.1 *Loss function*

For point processes, given the observation of all events in the interval $[0, T]$, we use the log-likelihood as in the equation below:

$$\mathscr{L} = \sum_{i:t_i \leq T} \log \lambda_{k_i}(t_i) - \underbrace{\int_{t=0}^{T} \lambda(t) dt}_{\text{Let us call it } \Lambda}. \tag{2.6}$$

We will derive this function following Mei e Eisner (2017). First, we define $N(t) = |\{h : t_h \leq t\}|$ as the count of events (of any type) before time t. Given the past history $\mathcal{H}_i$, the number of events in $(t_{i-1}, t]$ is $\Delta N(t_{i-1}, t) \stackrel{\text{def}}{=} N(t) - N(t_{i-1})$.

Let $T_i > t_{i-1}$ be the random variable of the time of the next event and K_{i+1} the random variable of the type of the next event. The cumulative distribution function and the probability density function of T_i (conditioned on $\mathcal{H}_i$) are given by:

$$F(t) = P(T_i \leq t) = 1 - P(T_i > t) \tag{2.7}$$

$$= 1 - P(\Delta N(t_{i-1}, t) = 0) \tag{2.8}$$

$$= 1 - \exp\left(- \int_{t_{i-1}}^{t} \lambda(s) ds \right) \tag{2.9}$$

$$= 1 - \exp(\Lambda(t_{i-1}) - \Lambda(t)) \tag{2.10}$$

$$f(t) = exp(\Lambda(t_{t-1}) - \Lambda(t))\lambda(t), \tag{2.11}$$

where $\Lambda(t) = \int_0^t \lambda(s) ds$ and $\lambda(t) = \sum_{k=1}^{K} \lambda_k(t)$.

In addition, given the past history $\mathcal{H}_i$ and the time of the next event t_i, the distribution of the event type k_i (probability of being of type k_i) is given by:

$$P(K_i = k_i \mid t_i) = \frac{\lambda_{k_i}(t_i)}{\lambda(t_i)}. \tag{2.12}$$

Therefore, we can derive the likelihood function L as follows:

$$L = \prod_{i:t_i \le T} L_i \tag{2.13}$$

$$= \prod_{i:t_i \le T} \{f(t_i) P(K_i = k_i \mid t_i)\} \tag{2.14}$$

$$= \prod_{i:t_i \le T} \{\exp(\Lambda(t_{i-1}) - \Lambda(t_i)) \lambda_{k_i}(t_i)\}. \tag{2.15}$$

Finally, by applying the logarithm to the likelihood we turn the product into a sum:

$$\mathscr{L} \stackrel{\text{def}}{=} \log L \tag{2.16}$$

$$= \sum_{i:t_i \le T} \log \lambda_{k_i}(t_i) - \sum_{i:t_i \le T} (\Lambda(t_i) - \Lambda(t_{i-1})) \tag{2.17}$$

$$= \sum_{i:t_i \le T} \log \lambda_{k_i}(t_i) - \Lambda(T) \tag{2.18}$$

$$= \sum_{i:t_i \le T} \log \lambda_{k_i}(t_i) - \int_{t=0}^{T} \lambda(t) dt. \tag{2.19}$$

The same log-likelihood connects all the processes we have seen. Equation (2.19) is not written for one particular process. It works for any temporal point process, and the only thing that changes from one process to another is the intensity $\lambda_k(t)$ that we put inside it.

Let us look at the easiest case first: a homogeneous Poisson process with a single event type and a constant rate λ. Since λ is the same everywhere, two things become easy. The sum of $\log \lambda$ over the n events is simply $n \log \lambda$ (we add the same number n times), and the integral of λ from 0 to T is simply λT (the area of a rectangle of height λ and width T). Substituting these two results into Equation (2.19) leaves the log-likelihood as

$$\log L = n \log \lambda - \lambda T,$$

and taking the exponential of both sides recovers the likelihood itself,

$$L = \lambda^n e^{-\lambda T}.$$

This looks just like the Poisson formula of Equation (2.4) (with $\Lambda = \lambda T$), which is $(\lambda T)^n e^{-\lambda T}/n!$: both are a rate raised to the number of events, times e to the minus the expected count. The two

differ only by a constant, because our likelihood describes the exact times of the events, while Equation (2.4) only describes how many events happened.

The Hawkes process uses this same equation, but now the constant λ is replaced by the history-dependent intensity $\lambda(t \mid \mathcal{H}_t)$ of Equation (2.20). Training a Hawkes model just means looking for the parameters that make this likelihood as large as possible, which is the same as making the negative log-likelihood (NLL) as small as possible. In the end, the parametric kernel, the neural networks, and the attention mechanisms in this work are all just different ways of computing $\lambda(t \mid \mathcal{H}_t)$. The way we compare them is always the same log-likelihood of Equation (2.19).

2.4 Hawkes processes

Poisson processes are a basic way of working with sequences of discrete events; however, they start from the assumption that such events are independent of one another. Even in a multivariate and non-homogeneous Poisson process, where the variation in the probability of an event occurring at t varies as a function of t, the events are still independent of one another. It assumes that an event of type k happens in the infinitesimal interval $[t, t+\mathrm{d}t]$ with probability $\lambda_k(t)\mathrm{d}t$. The total intensity of all events is given by $\lambda(t) = \sum_{k=1}^{K} \lambda_k(t)$.

The Hawkes process, on the other hand, starts from the principle that past events influence the probability of future events. In a traditional Hawkes process, we assume that this probability is always increased, added to the probability from past events, and decays exponentially (MEI; EISNER, 2017).

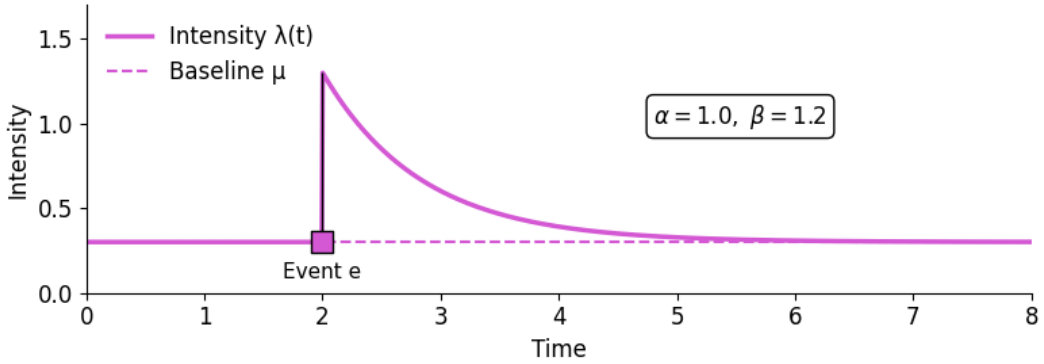
The Hawkes process is considered doubly stochastic (ZUO *et al.*, 2021), because $\lambda(t)$ is a random variable, as in a Cox process, but it is also a Poisson process. The intensity of a traditional univariate Hawkes process is given by:

$$\lambda(t \mid \mathcal{H}_t) = \mu + \sum_{t_i < t} \alpha e^{-\beta(t-t_i)}, \tag{2.20}$$

where α represents the instantaneous increase in the probability intensity caused by event e and β represents the speed at which this intensity decays over time.

Figure 2 illustrates the probability intensity λ of an event where, from $t = 0$ to $t = 2$, this intensity stays at a baseline μ. At time $t = 2$, event e happens, raising the probability intensity instantaneously. Then, λ decays exponentially, returning to the stationary state μ, which represents the absence of influence from any event at time t.

Figure 2 – Probability intensity function representing instantaneous self-excitation and exponential decay.



Throughout this work we will consider the multivariate Hawkes process, which is able to capture the influence of events of different types k on one another, such as a post on a social network positively influencing the occurrence of another type of interaction, or a neurological event being a trigger for a certain stroke.

2.4.1 Relationship with the Cox process

We can place the Hawkes process within a progression of increasingly flexible intensity models. In the homogeneous Poisson process, the intensity λ is constant. In the non-homogeneous Poisson process, it now varies in time in a deterministic way, $\lambda(t)$. One step further is the **Cox process**, also called the doubly stochastic Poisson process, in which the intensity $\lambda(t)$ itself is a random process, governed by factors external to the event history.

The Hawkes process can be seen as a particular case of this idea of stochastic intensity: the randomness of $\lambda(t)$ does not come from an external source, but from the past events themselves, each one temporarily raising the probability of new events (Equation (2.20)). It is in this sense that the Hawkes process is described as doubly stochastic, as mentioned earlier. In this work, it is exactly this dependence on the history (and the exponential decay of the influence of past events) that motivates the recency bias of the proposed model.

Table 1 organizes this progression. It contrasts the four models by the form of their intensity, by whether the intensity depends on the past events, by whether the counts in disjoint intervals are independent, and by whether the process keeps memory of earlier events. The three Poisson-based models share the assumption of independent increments and differ only in how the rate varies. The Hawkes process breaks this assumption: its intensity is a function of the history $\mathcal{H}_t$, so an event raises the probability of future events, and the counts in disjoint intervals

are no longer independent. This move from independent increments to history dependence is what makes the Hawkes process able to represent self-excitation, and it is the property that the models in this work try to capture.

Table 1 – Taxonomy of point processes, from the Poisson process to the Hawkes process.

Process	Intensity	Depends on history	Memory of past events
Homogeneous Poisson	λ (constant)	No	No
Non-homogeneous Poisson	$\lambda(t)$ (deterministic)	No	No
Cox (doubly stochastic)	$\lambda(t)$ (random, external)	No	External source
Hawkes	$\lambda(t \mid \mathcal{H}_t)$	Yes	Yes (decay kernel)

2.5 Neural temporal point processes

We saw that Hawkes processes describe many scenarios; however, their classical form does not account for the everyday complexities that are intrinsic to many events. Events may have an inhibitory influence, instead of an excitatory one, on other events, such as, for example, preventive maintenance lowering the probability of a failure in a piece of equipment. Another assumption of traditional Hawkes processes is to consider that events of the same type have a constant influence on other events; however, we know that, for example, when watching a phenomenon for the twentieth time our brain is less excited than when we witness the event for the first time. Finally, we can also recall phenomena where the influence on other events does not decay in an exponential way (MEI; EISNER, 2017).

Mei e Eisner (2017) mention how hard it is to apply the traditional Hawkes process in scenarios with missing data. Even in domains where Hawkes would be appropriate, the nature of the data itself may present challenges, such as partially observed sequences or data omitted in a systematic way. There is also data omitted in a stochastic way, which can be quite complex for the traditional version of Hawkes.

To address such limitations of the Hawkes process, approaches using neural networks were proposed, with the work of Mei e Eisner (2017) being one of the most widespread and accepted by the community, where they proposed the NHP.

The first step was to relax the rule of keeping α and μ strictly positive, and this allowed the values to range over $\mathbb{R}$, addressing inhibition ($\alpha < 0$) and inertia ($\mu < 0$). However, the result can now be negative, which would not make sense for a probability intensity, so it is necessary to apply a transformation $f = \mathbb{R} \to \mathbb{R}_+$, with $\lambda_k(t) = f_k(\tilde{\lambda}_k(t))$, where k is the event

type. The ReLU function $f(x) = \max(x, 0)$ would be a natural choice for this conversion, but it takes the value 0 for negative numbers, and our intensity needs to take always positive values whenever there is a possibility of an event occurring.

The function then chosen by the authors was the softplus $f(x) = s \cdot \log(1 + exp(x/s))$, where s is a learned scale parameter that brings numerical stability. Without it, when the unit of x varies, for example from seconds to milliseconds, the base intensity $f(\mu_k)$ becomes a thousand times smaller. This forces μ_k to take negative values and creates a strong inertia effect.

The second step was to remove the restriction that past events have independence and additive influence on $\lambda(t)$. Instead of predicting $\lambda(t)$ using the summation of the traditional parametric Hawkes model (Equation (2.20)), Mei e Eisner (2017) used a recurrent neural network, which allowed them to capture complex patterns that were previously hard to perceive in parametric versions of the model.

As in the traditional models, each event type k has a time-dependent intensity $\lambda_k(t)$, which jumps at each new event and decays exponentially toward a stationary value, analogous to the μ in Equation (2.20). However, in the model proposed by Mei e Eisner (2017) this dynamic is controlled by a hidden state vector $\mathbf{h}(t) \in (-1, 1)^D$, which depends on the vector $\mathbf{c}(t) \in \mathbb{R}^D$, which is the memory vector of a recurrent neural network with one layer and D neurons.

Mei e Eisner (2017) proposed a continuous-time variation of the *Long Short-Term Memory* (LSTM), the Continuous-Time LSTM (CT-LSTM), where after each event each memory cell c decays exponentially at a certain rate δ toward a stable (stationary) state $\bar{\mathbf{c}}$. At each moment $t > 0$, it is possible to obtain the intensity of each event type with $\lambda_k(t) = f_k(\mathbf{w}_k^T \mathbf{h}(t))$, where $\mathbf{h}$ can be computed with $\mathbf{h}(t) = \mathbf{o}_i \odot (2\sigma(2\mathbf{c}(t)) - 1$ for $t \in (t_{i-1}, t_i]$, where $\mathbf{o}$ is the LSTM output vector and $\mathbf{w}$ a weight vector. At each new event t_i, the CT-LSTM updates its gates and memory cell through the following equations:

$$\mathbf{i}_{i+1} \leftarrow \sigma(\mathbf{W}_i \mathbf{k}_i + \mathbf{U}_i \mathbf{h}(t_i) + \mathbf{d}_i) \tag{2.21}$$

$$\mathbf{f}_{i+1} \leftarrow \sigma(\mathbf{W}_f \mathbf{k}_i + \mathbf{U}_f \mathbf{h}(t_i) + \mathbf{d}_f) \tag{2.22}$$

$$\mathbf{z}_{i+1} \leftarrow 2\sigma(\mathbf{W}_z \mathbf{k}_i + \mathbf{U}_z \mathbf{h}(t_i) + \mathbf{d}_z) - 1 \tag{2.23}$$

$$\mathbf{o}_{i+1} \leftarrow \sigma(\mathbf{W}_o \mathbf{k}_i + \mathbf{U}_o \mathbf{h}(t_i) + \mathbf{d}_o) \tag{2.24}$$

$$\mathbf{c}_{i+1} \leftarrow \mathbf{f}_{i+1} \odot \mathbf{c}(t_i) + \mathbf{i}_{i+1} \odot \mathbf{z}_{i+1} \tag{2.25}$$

$$\bar{\mathbf{c}}_{i+1} \leftarrow \bar{\mathbf{f}}_{i+1} \odot \bar{\mathbf{c}}_i + \bar{\mathbf{i}}_{i+1} \odot \mathbf{z}_{i+1} \tag{2.26}$$

$$\mathbf{o}_{i+1} \leftarrow f(\mathbf{W}_d \mathbf{k}_i + \mathbf{U}_d \mathbf{h}(t_i) + \mathbf{d}_d) \tag{2.27}$$

Equations (2.21), (2.22), and (2.24) refer respectively to the *input*, *forget*, and *output gates*, common to a conventional LSTM network. Equation (2.23) refers to the candidate value (*Candidate Memory*). Equations (2.25) and (2.26) represent the Memory Cell (*Memory Cell*) and the stable state of the Memory Cell. Finally, Equation (2.27) computes the vector δ, the decay rate, analogous to the β in Equation (2.20).

The equations above are quite similar to a conventional (discrete) LSTM, but with two fundamental differences. The updates do not depend on the previous value t_{t-1}, but on the value of $\mathbf{h}(t_i)$, after it has undergone decay over the period $t_i - t_{i-1}$. In addition, Equations (2.26) and (2.27) are new, added by Mei e Eisner (2017), and they define how, over time, as $t > t_i$ increases, the elements of $\mathbf{c}(t)$ will continuously decay from $\mathbf{c}_{i+1}$ to $\bar{\mathbf{c}}_{i+1}$. $\mathbf{c}(t)$ is computed according to the equation

$$\mathbf{c}(t) \overset{\text{def}}{=} \bar{\mathbf{c}}_{i+1} + (\mathbf{c}_{i+1} - \bar{\mathbf{c}}_{i+1}) \exp(-\delta_{i+1}(t - t_i)) \quad \text{for } t \in (t_i, t_{i+1}]. \tag{2.28}$$

Recall that $\mathbf{c}(t)$ is used in the function $\mathbf{n}(t)$, which is analogous to the LSTM hidden state vector $\mathbf{h}_i$, which in turn is used in the computation of $\lambda_k(t)$, for a given event type k.

The approach of Mei e Eisner (2017) was simple but quite clever, achieving excellent results on both synthetic and real datasets, often still being the best benchmark, despite being a 2017 algorithm.

2.6 Transformers and the attention mechanism

Approaches based on recurrent neural networks (RNN), such as the Continuous-Time LSTM (CT-LSTM) proposed by Mei e Eisner (2017), brought great advances in the modeling of complex patterns; however, they kept the weak point of RNNs, which is the fact that their processing is sequential. RNNs process one event after another, that is, the computation of the next event (t_{i+1}) depends on the current event (t_i), which makes training the network slow. In some scenarios, with very long sequences, it becomes infeasible to use an RNN. More recent works adapted the Transformer architecture proposed by Vaswani *et al.* (2017), adapting the attention mechanism for TPP, especially aimed at the Hawkes process, which is the focus of this work. The following subsections will help us recall how the architecture proposed in 2017 works.

2.6.1 *Encoders and Decoders*

The Encoder is the component responsible for processing the input sequence and transforming it into a representation richer in information. It is composed of N identical stacked layers (in the original model of Vaswani *et al.* (2017), $N = 6$ and the dimension of the vectors is $d_{model} = 512$, but these values are hyperparameters that vary according to the application). Each layer has two sublayers: the first is the Multi-Head Self Attention, which computes the relevance of each input with respect to the other inputs in the sequence. The second is a fully connected feed-forward network, which applies nonlinearity at each position.

Similarly, the Decoder is also made up of N stacked layers. However, to generate the output, the Decoder uses a third sublayer, which performs a Multi-Head Attention over the Encoder output, connecting the context processed by the Encoder with the sequence generated by the Decoder. In addition, the Decoder uses a mask to prevent the model from computing future positions in the sequence, so that each prediction considers only known events (VASWANI *et al.*, 2017).

2.6.2 *Attention mechanism*

The main innovation introduced by the model is the attention mechanism, also called Scaled Dot-Product Attention, without the need for recurrent neural networks, which makes all the inputs (events) be processed simultaneously. In this way the model is able to compute the influence (attention) of each event with respect to the other events.

Unlike RNNs, which keep only a single hidden state vector $\mathbf{h}$, Transformer-based networks compute the influence of each event and compute $\mathbf{h}$ through a weighted sum, which in turn was computed based on the influence of each event, within the context of the list of events.

The input consists of query vectors $\mathbf{Q}$ and key vectors $\mathbf{K}$ of dimension d_k and value vectors $\mathbf{V}$ of dimension d_v. The product of the query with all the keys is then computed, each divided by $\sqrt{d_k}$, and then a softmax function is applied to obtain the weights of the values (of the key-value vectors). The output matrix is computed as

$$\text{Attention}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{softmax}(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}})\mathbf{V}. \tag{2.29}$$

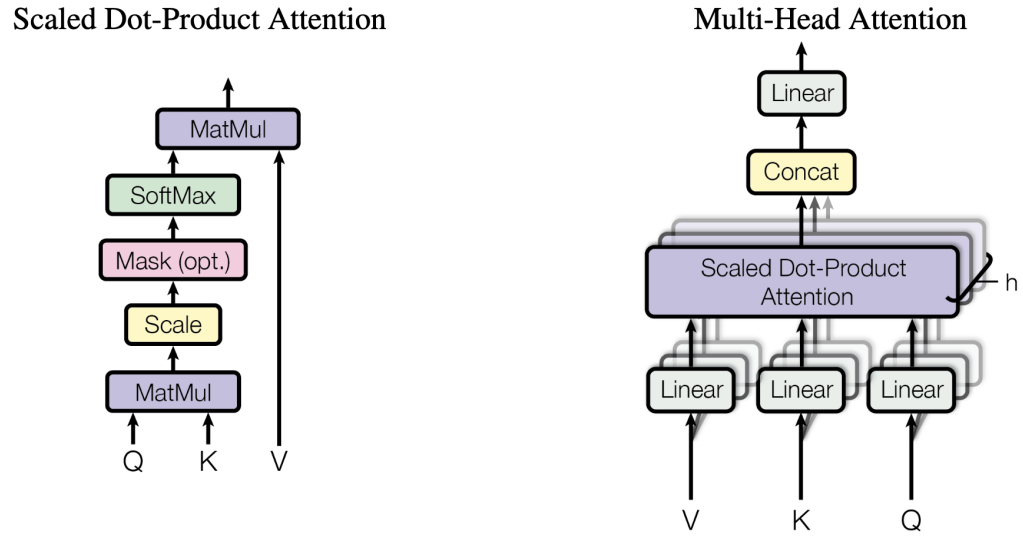
2.6.3 *Multi-Head Attention*

Instead of performing a single Attention function with Queries, Keys, and Values of dimension d_{model}, Vaswani *et al.* (2017) decided to linearly project the **Q**, **K**, and **V** vectors a number h of times with different, learned linear projections to d_k, d_k, and d_v dimensions, respectively. For each of the projected versions of **Q**, **K**, and **V** the Attention function is run in parallel, producing outputs of d_v dimensions. The outputs are then concatenated and projected again, producing the final values, as shown in the equation below:

$$\begin{aligned} &\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1,\ldots,\text{head}_h)W^O, \\ &\qquad\qquad \text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V). \end{aligned} \tag{2.30}$$

Figure 3 illustrates the difference between the simple attention mechanism and the multi-head attention mechanism (VASWANI *et al.*, 2017).

Figure 3 – Attention x Multi-Head Attention.



Source: Vaswani *et al.* (2017).

3 RELATED WORK

This chapter reviews the Transformer-based point process models that precede and provide the basis for the HoTHP, along with the length extrapolation techniques that motivate its proposal. Here we start from the Transformer Hawkes Process (THP) and the RoTHP, which replaces the absolute temporal *encoding* with *rotary position embeddings*. Finally, we discuss the problem of *length extrapolation* and the notion of a recency inductive bias, which connects these models to the Hawkes kernel and gives rise to the approach proposed in this dissertation.

3.1 Transformer Hawkes Process (THP)

The Transformer architecture (VASWANI *et al.*, 2017) showed that using Attention can offer good advances over the use of recurrent neural networks; however, like traditional RNNs, Transformers were also designed for discrete events, such as the words of a sentence. Zuo *et al.* (2021) had to modify the architecture, turning it into a continuous one, just as Mei e Eisner (2017) had to do when turning the traditional LSTM into a Continuous Time-LSTM.

In translation problems, which were originally the first application of Transformers, the order of the words is enough, with no need for precision beyond that. However, in the analysis of point processes we need to know the moment at which each event occurs. To solve this problem, Zuo *et al.* (2021) proposed a temporal encoding, represented by Equation (3.1), where the argument is not the (discrete) position, as proposed by Vaswani *et al.* (2017), but the timestamp t_j:

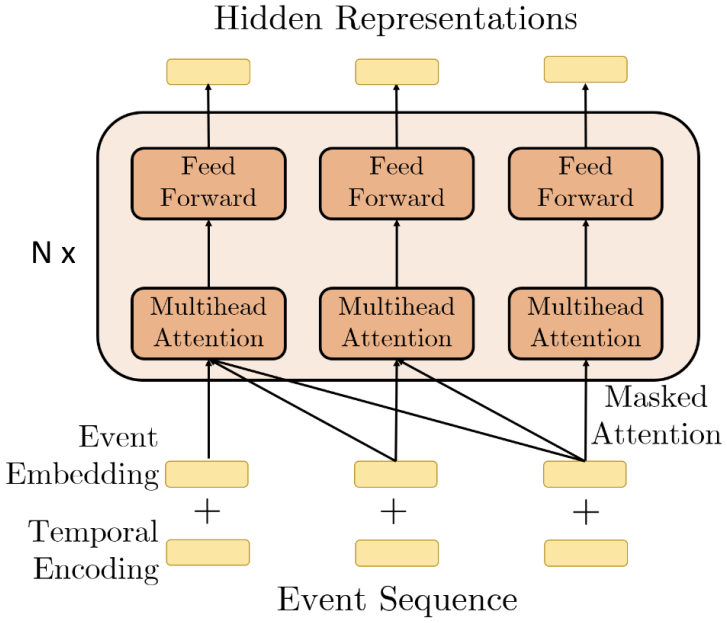
$$[z(t_j)]_i = \begin{cases} \cos\left(\frac{t_j}{10000^{\frac{i-1}{M}}}\right), & \text{if } i \text{ is odd,} \\ \sin\left(\frac{t_j}{10000^{\frac{i}{M}}}\right), & \text{if } i \text{ is even.} \end{cases} \tag{3.1}$$

Equation (3.1) uses trigonometric functions to determine a temporal encoding for each timestamp, that is, for each t_j a deterministic $\mathbf{z}(t_j) \in \mathrm{R}^M$ is computed, where M is the size of the encoding dimension. Besides the temporal encoding, an embedding layer, the matrix $\mathbf{U}$, is also used for the different event types. The embeddings are then represented by $\mathbf{X} = (\mathbf{UY} + \mathbf{Z})^T$, where $\mathbf{Y}$ is the one-hot encoding collection of event types and $\mathbf{Z}$ the matrix with the $\mathbf{z}$ vectors concatenated. After the embedding layers, $\mathbf{X}$ is passed through the same self-attention mechanism of Equation (2.29), now with $\mathbf{Q} = \mathbf{XW}^Q$, $\mathbf{K} = \mathbf{XW}^K$, and $\mathbf{V} = \mathbf{XW}^V$, which produces an output

S. This output then passes through a feed forward network to produce $\mathbf{h}(t)$:

$$\mathbf{H} = \mathrm{ReLU}(\mathbf{S}\mathbf{W}_1^{FC} + \mathbf{b}_1)\mathbf{W}_2^{FC} + \mathbf{b}_2, \quad \mathbf{h}(t_j) = \mathbf{H}(j,:). \tag{3.2}$$

Figure 4 – Architecture of the Transformer Hawkes Process.



Source: Zuo *et al.* (2021).

Figure 4 shows the architecture of the Transformer Hawkes Process. Each event sequence passes through two embedding layers and several times through multi-head self attention modules.

Since it models a multivariate process, the total intensity at a time t is the sum of the intensities of all K event types:

$$\lambda(t \mid \mathscr{H}_t) = \sum_{k=1}^{K} \lambda_k(t \mid \mathscr{H}_t). \tag{3.3}$$

As we saw earlier, after the whole process of embedding, attention, and feed-forward network, $\mathbf{H}$ is finally produced, which lets us compute λ_k, needed in Equation (3.3). The intensity computation for each event type is given by:

$$\lambda_k(t \mid \mathscr{H}_t) = f_k\left(\underbrace{\gamma_k \frac{t - t_j}{t_j}}_{\text{current}} + \underbrace{\mathbf{w}_k^\top \mathbf{h}(t_j)}_{\text{history}} + \underbrace{b_k}_{\text{inertia}}\right). \tag{3.4}$$

In Equation (3.4), the first "current" part is an interpolation between t_j and t_{j+1} where γ_k, a learned element, modulates the importance of this interpolation. We write this coefficient as

γ_k, which the original THP denotes α_k, to avoid confusion with the α of the Hawkes process (Equation (2.20)), which plays a different role. The second part uses the history $\mathbf{h}(t)$ of the events, and the last element b_k represents the stationary state, where the probability of the next event is not influenced by the event history.

Zhang *et al.* (2020), with the *Self-Attentive Hawkes Process* (SAHP), proposed a different computation for the temporal encoding. Unlike the THP, which uses only the timestamp t_i, the SAHP shifts the event to a new position i' using both the position i and the timestamp t_i. This is computed with $\sin(\omega_k \times i + w_k \times t_i)$, where ω is the angular frequency and w is a scale parameter, adjusted by the model, that converts the timestamp t_i into a phase shift inside the sinusoidal function. As in other models, the Self-Attentive Hawkes Process uses sine for even events and cosine for odd events.

3.2 Rotary Position Embeddings (RoPE) and the RoTHP

We saw that the use of RNNs has limitations both for the memory of distant events and in the training of the network itself, because each event needs to be processed before the next event, since it carries the state of the previous event. Through the attention mechanism, models using Transformers solve these two problems; however, most implementations are not resilient to noise in the data (timestamps). In real datasets, the presence of such noise is very common, mainly due to physical delays in recording the timestamp. For example, in wireless networks, devices record events even without having their clocks perfectly synchronized with one another, which causes discrepancies in the final dataset. The THP and its most common variants adopt a sinusoidal temporal encoding, ignoring such characteristics of the data.

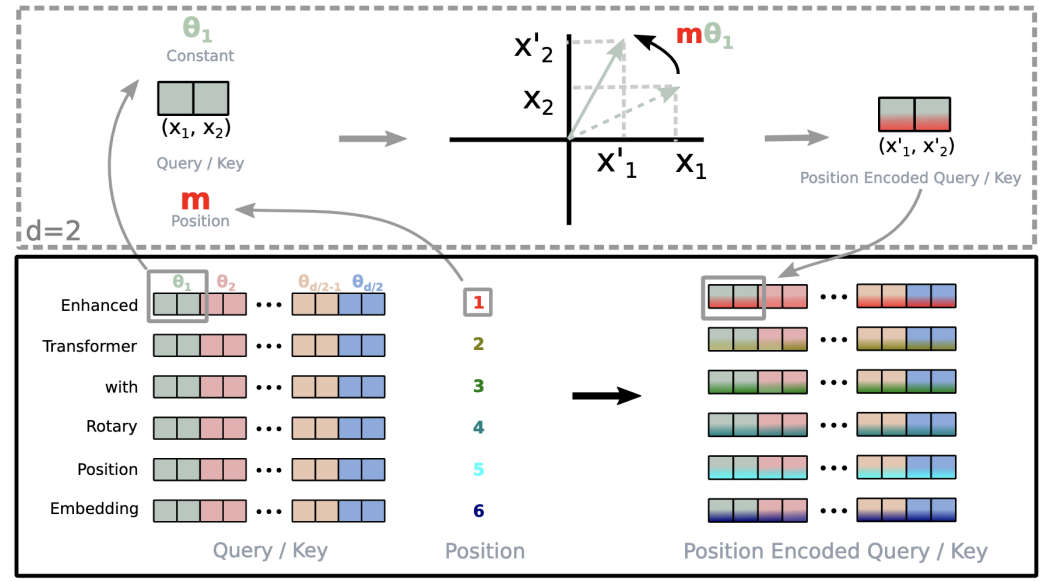
Su *et al.* (2023) showed that the likelihood computation uses only the difference between the event times and, in this way, proposed to use the relative distances between tokens in language models, replacing the temporal encoding of the traditional Transformer architecture with the relative time, inside the attention mechanism. To do this, we use the input vector (embedding) in pairs $(d_i,\ d_{j+1})$. For a token at position m in the sequence, each pair passes through a 2D rotation, producing the rotation matrix $\mathbf{R}^d_{\Theta,m}$, with angles $\Theta = \{\theta_i = 10000^{-2(i-1)/d},\ i \in [1,2,\ldots,d/2]\}$. The rotated Query and Key are given by $f_{q,k}(\mathbf{x}_m, m) = \mathbf{R}^d_{\Theta,m}\mathbf{W}_{q,k}\mathbf{x}_m$, where $\mathbf{x}_m$ is the embedding of the token at position m, and q and k denote the Query and Key components

of the attention mechanism:

$$\mathbf{R}^{d}_{\Theta,m} = \begin{pmatrix} \cos(m\theta_1) & \sin(m\theta_1) & 0 & 0 & \cdots & 0 & 0 \\ -\sin(m\theta_1) & \cos(m\theta_1) & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos(m\theta_2) & \sin(m\theta_2) & \cdots & 0 & 0 \\ 0 & 0 & -\sin(m\theta_2) & \cos(m\theta_2) & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos(m\theta_{d/2}) & \sin(m\theta_{d/2}) \\ 0 & 0 & 0 & 0 & \cdots & -\sin(m\theta_{d/2}) & \cos(m\theta_{d/2}) \end{pmatrix}.$$

Figure 5 illustrates the tokens of a language model, the position vector, and how RoPE uses pairs of the embedding to apply rotation. The rotation matrix $\mathbf{R}^{d}_{\Theta,m}$ is then applied to the Query and Key matrices $\mathbf{Q}$ and $\mathbf{K}$ of the attention mechanism, which is exactly the transformation $f_{q,k}$ defined above.

Figure 5 – Rotary Position Embeddings (RoPE).



Source: Su *et al.* (2023).

Dai *et al.* (2026) adapted RoPE to point processes, presenting the RoTHP. This consisted of no longer using the index position of the token of a language model, but the timestamp. A great benefit of this approach was making the likelihood time-invariant, since it deals only with the relative times (deltas). This means that if we have a time series $\{t_1+\sigma, t_2+\sigma, \ldots, t_n+\sigma\}$ it will produce the same result as $\{t_1, t_2, \ldots, t_n\}$, and this solves a consistency problem inherent to models that use absolute times, allowing a time shift.

The metrics presented in Tables 2, 3, and 4 show a clear superiority of the RoTHP over several traditional models on several datasets. Although the accuracy is competitive with the THP, the advance was significant in the log-likelihood computation (NLL) and in the RMSE. In the RMSE the improvement was substantial, especially on the Stack Overflow (SO) datasets, where the RoTHP surpassed the THP, which until then was the best model, by a significant margin. This suggests that the proposed embedding rotation model captures patterns more precisely than the traditional sinusoidal encoding.

In addition, part of the robustness of the RoTHP comes from its ability to remain time-invariant. By replacing absolute timestamps with time intervals through rotation in the 2D plane, the model effectively managed to mitigate the impact of noise, often caused by synchronization problems present in real datasets. This shows that adapting RoPE to point processes increased the generalization capability of current models that use Transformers. As a result, the RoTHP establishes a new state of the art for point process benchmarks, as far as we know.

Table 2 – Model log-likelihood by dataset.

Models	Financial	SO	Synthetic	Retweet	MemeTrick	Mimic-II
RMTPP	-3.89	-2.6	-1.33	-5.99	-6.04	-1.35
NHP	-3.6	-2.55	–	-5.6	-6.23	-1.38
SAHP	–	-1.86	0.59	-4.56	–	-0.52
THP	-1.11	-0.039	0.791	-2.04	0.68	0.48
RoTHP	**1.076**	**0.389**	**1.01**	**2.01**	**1.71**	**0.64**

Table 3 – Accuracy comparison (in %).

Models	Financial	Mimic-II	SO
RMTPP	61.95	81.2	45.9
NHP	62.20	83.2	46.3
THP	62.23	84.9	**46.4**
RoTHP	**62.26**	**85.5**	46.33

3.3 Length Extrapolation and inductive bias

A common limitation shared by many Transformer-based sequence models is their inability to generalize to sequences longer than those used during training. This problem is known as *length extrapolation*. Given a model trained on sequences of size L_{train}, we check

Table 4 – RMSE comparison.

Models	Financial	Mimic-II	SO
RMTPP	1.56	6.12	9.78
NHP	1.56	6.13	9.83
SAHP	–	3.89	5.57
THP	0.93	0.82	4.99
RoTHP	**0.60**	**0.57**	**1.33**

whether it remains stable when tested with sequences of size $L_{test} > L_{train}$ (PRESS *et al.*, 2021). This limitation matters in practice. In many real-world scenarios, collecting long sequences is expensive or even impractical and, therefore, it is desirable to train on shorter sequences and run the model on arbitrarily longer sequences.

The main cause of this failure has been attributed to the *positional encoding* strategy applied in traditional Transformer models. The standard sinusoidal embedding, as introduced by Vaswani *et al.* (2017), produces absolute representations of the positions. When the model encounters a position index that was never presented during training, these representations are effectively considered *out of distribution*, causing a rapid degradation in performance, as we show in Chapter 5.

Press *et al.* (2021), in the setting of language models, showed empirically that sinusoidal models are unable to extrapolate to sequences larger than the ones used in training, with the perplexity metric (a measure specific to language modeling) increasing dramatically as L_{test} grows, presenting a model called *Attention with Linear Biases* (ALiBi), which we detail in Section 3.3.1. Rotational embeddings, such as RoPE (SU *et al.*, 2023), present improvements over the sinusoidal encoding by using relative encoding instead of absolute positions, however, Press *et al.* (2021) showed that even RoPE fails to extrapolate to substantially larger sequences, as shown in Figure 6.

3.3.1 Attention with Linear Biases (ALiBi)

Press *et al.* (2021) proposed a very different approach to positional encoding. Instead of adding or rotating embeddings, they introduced a static, non-learnable penalty directly into the attention score. This method, called *Attention with Linear Biases (ALiBi)*, replaces the positional embedding step completely. The main motivation of the model is that it should give less importance to tokens as they move away from the current index, which the authors call *inductive bias towards recency*. Instead of learning this property from the data, ALiBi computes

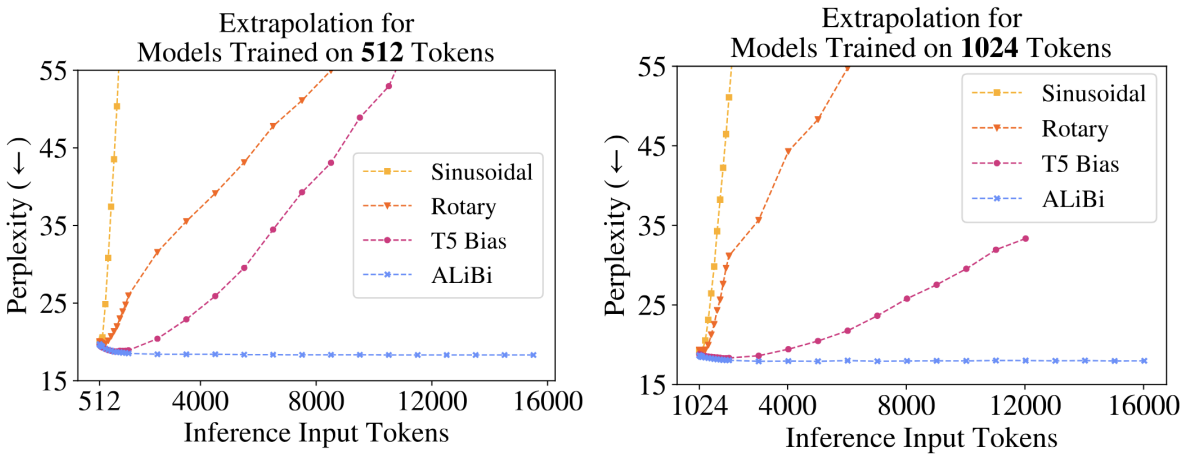


Figure 6 – As the extrapolation increases, we clearly see that the degradation shown by RoPE is smaller than that of traditional sinusoidal methods, but still much sharper than the model proposed by Press *et al.* (2021)

Source: Press *et al.* (2021).

this decay of "importance" in its own architecture.

In traditional Transformer models, position embeddings are added to the embeddings of the words (tokens). The attention weights $\mathbf{a}_i$ of a query i are obtained by applying the softmax to the dot products between $\mathbf{q}_i$ and all the keys $\mathbf{K}$:

$$\mathbf{a}_i = \text{softmax}\left(\mathbf{q}_i \mathbf{K}^\top\right), \tag{3.5}$$

where these weights are then multiplied by the values $\mathbf{V}$. ALiBi only modifies the step after the dot product, leaving everything else intact:

$$\mathbf{a}_i = \text{softmax}\left(\mathbf{q}_i \mathbf{K}^\top + m \cdot [-(i-1), \ldots, -2, -1, 0]\right), \tag{3.6}$$

where the vector $[-(i-1), \ldots, -2, -1, 0]$ represents the distances of each position K to the current Q position i, and the scalar m is a slope coefficient specific to each attention head, fixed before training and never updated. The negative sign guarantees that the penalty lowers the attention score as the distance between Q and K increases, effectively suppressing the attention to distant tokens. Here, n is the number of attention heads, and the n slopes (one for each head) form a geometric progression whose ratio and first term are both $2^{\frac{-8}{n}}$. For example, with $n = 8$ heads the slopes are $\frac{1}{2}, \frac{1}{4}, \ldots, \frac{1}{256}$ (PRESS *et al.*, 2021).

3.3.2 Inductive bias and its relevance to the Hawkes process

An inductive bias is an assumption that guides the model, independently of the trained data. In the positional context of ALiBi, we can state that the inductive bias dictates that

distant words/tokens are less relevant, linearly, than closer ones. The penalty is applied directly in the code, and not only in the learned data.

The inductive bias is not arbitrary. In fact, it has been present in the Hawkes process since its proposal, in 1971. Looking at the Hawkes process equation, the influence of a past event t_i decays as the interval $t - t_i$ increases, in proportion to β. The greater the distance, the smaller the contribution of the event to the process. This shows that the Hawkes process has at its core an inductive bias represented by the exponential decay parameter.

The THP and the RoTHP replace the parametric Hawkes kernel with an attention mechanism. This brought enormous advantages to these models, such as the possibility of capturing more complex relationships, and even temporal invariance, in the case of the RoTHP, but the price paid was moving away from the formal equation of the original Hawkes process.

The attention mechanism does not impose any constraint on how it should consider the distance between the tokens, in the case of a TPP, between the events. Any decay that emerges from the model is the result of the learning performed with the data and not a structural guarantee. In fact, due to the oscillatory nature of the sinusoidal computation, occasionally distant events may receive more relevance than close events.

In short, the THP and the RoTHP traded the structural decay of the Hawkes process for the flexibility of attention, but attention gives no guarantee of decay, which is what fails under length extrapolation. The next chapter (Chapter 4) presents the HoTHP, our proposal that brings this decay back into the attention through a hyperbolic kernel.

4 METHODOLOGY

The HoTHP (*Hyperbolic Rotary Position Embedding-based Transformer Hawkes Process*) is a variant of the THP that replaces the trigonometric kernel of the RoTHP with a hyperbolic kernel that has exponential decay, inspired by the HoPE (DAI *et al.*, 2025). The main motivation is length extrapolation, where we train on short sequences and keep performance stable on longer sequences. The mechanism behind this ability is an exponential recency bias embedded directly in the attention kernel, which makes the scores decay monotonically with the temporal distance, reflecting the structure of the Hawkes process itself.

4.1 The oscillatory limitation of the RoTHP

As we saw in Section 3.2, the RoTHP represents the position of each event by a trigonometric rotation applied to the query $\mathbf{q}$ and key $\mathbf{k}$ vectors. This rotation is the positional embedding of the RoTHP: unlike the Transformer, it is not added to the input, it is the way temporal position enters the model, and the attention score it produces is the kernel we analyze in this section. The attention score between events i and j depends only on $\Delta t = t_i - t_j$, which is guaranteed by the algebraic properties of the trigonometric rotation matrix, which is orthogonal.

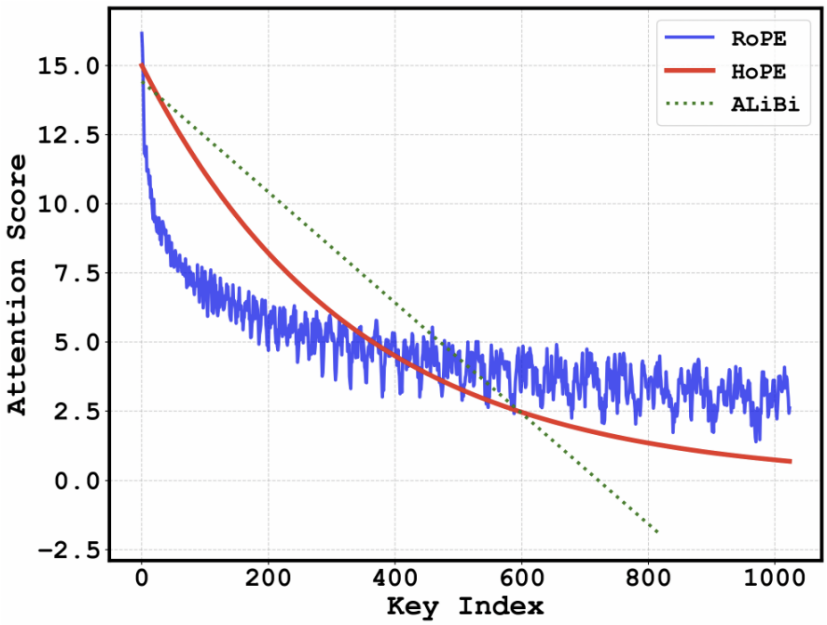
This trigonometric rotation was inherited from language models, where the positional encoding only needs to capture the order of the tokens. In a temporal point process, however, what matters is also the magnitude of the elapsed time, since the influence of a past event decays with the actual temporal distance. An encoding built for order does not naturally reproduce this magnitude-dependent decay.

The problem is that the resulting kernel is oscillatory. Sine and cosine are not monotonic functions. As a result, the attention score can grow for events more distant in time, depending on the phase of the oscillation, and shrink for nearby events. This directly contradicts the kernel of the Hawkes process, $\varphi(\Delta t) = \alpha e^{-\beta \Delta t}$, which is strictly decreasing.

In practice, the most serious consequence of this oscillation appears in length extrapolation. When tested on sequences longer than the training ones, the model encounters temporal differences Δt that it never observed during training. A kernel with exponential decay would have a predictable behavior in this region, with the influence continuing to fall. The trigonometric kernel, on the other hand, keeps oscillating, and a very distant event can receive a high score simply because the phase of the cosine fell on a peak at that Δt. The model did not learn to handle

these phases because it never saw them during training, and the kernel has nothing that forces the score to decay. It is this absence of a recency bias that prevents the RoTHP from extrapolating in a stable way. Figure 7 shows this contrast. The attention score of the RoPE decays only partially and keeps oscillating around a plateau, so distant keys are never fully suppressed. The hyperbolic kernel of the HoPE, by contrast, decays smoothly and monotonically toward zero.

Figure 7 – Attention score as a function of the relative distance between tokens (key index), for the RoPE, the HoPE, and the ALiBi.



Source: Dai *et al.* (2025).

This raises a natural question about the positional embedding itself: what period does it use, and would a longer one remove the oscillation? The period is that of the sinusoids that encode time, set by the RoPE frequencies $\theta_j = 10000^{-2(j-1)/d}$ (Equation (4.5)), the same frequencies that the additive sinusoidal encoding of the THP also uses. Note that 10000 is the base of the frequency formula, not a period: each component j oscillates with its own period $2\pi/\theta_j$, ranging from 2π (the fastest component, $\theta_1 = 1$) to about $2\pi \times 10000$ (the slowest). A longer period would not remove the oscillation, because this shortest period, $2\pi \approx 6.28$, never changes: the exponent for $j = 1$ is zero, so $\theta_1 = 1$ regardless of the base 10000. After the normalization of Equation (4.11), the average gap between events is about 1, so a sequence covers tens of events in training and a few hundred in extrapolation, far more than 6.28. The cosine then completes many cycles over a single sequence, so the oscillation is strong, not a small detail.

Changing the base does not help either. A larger base only shrinks the other frequencies and leaves $\theta_1 = 1$ untouched, so the shortest period stays at 2π. And if we forced θ_1 to be small enough to stretch the period past the longest distance seen in extrapolation (about 500), the kernel would become almost flat over the training sequences and could no longer tell nearby events from distant ones. In short, a longer period removes the oscillation but throws away the temporal resolution. The HoTHP avoids this trade-off: its decay is monotonic by construction ($\theta' > \max_j \theta_j$, Section 4.3), with no period to tune.

4.2 From the trigonometric kernel to the hyperbolic kernel

For each pair of dimensions $(2j-1, 2j)$, the RoTHP applies the 2×2 trigonometric rotation block:

$$\mathbf{R}_j(\Delta t) = \begin{bmatrix} \cos(\theta_j \Delta t) & \sin(\theta_j \Delta t) \\ -\sin(\theta_j \Delta t) & \cos(\theta_j \Delta t) \end{bmatrix}. \tag{4.1}$$

The HoPE (DAI *et al.*, 2025) proposes to replace this rotation with a hyperbolic rotation, whose corresponding block is:

$$\mathbf{B}_j(\Delta t) = \begin{bmatrix} \cosh(\theta_j \Delta t) & \sinh(\theta_j \Delta t) \\ \sinh(\theta_j \Delta t) & \cosh(\theta_j \Delta t) \end{bmatrix}. \tag{4.2}$$

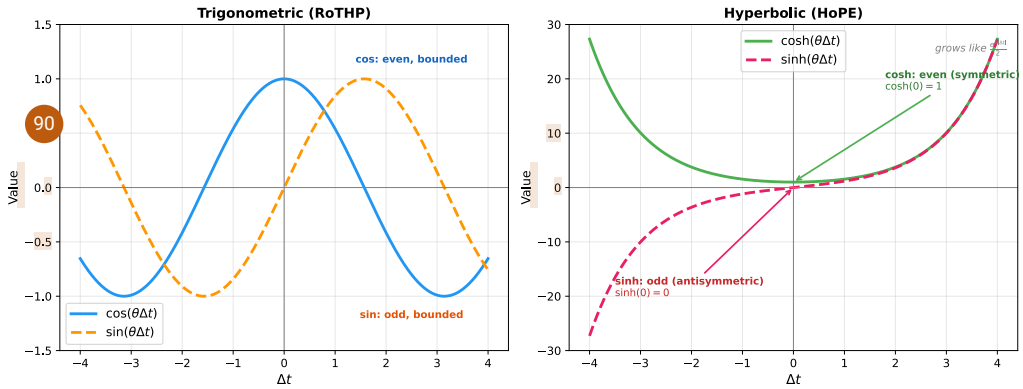
Unlike cos and sin, the functions cosh and sinh are monotonic for $\Delta t > 0$, removing the oscillations. However, we cannot use $\mathbf{B}_j$ directly because the hyperbolic rotation is not orthogonal and the inner product $\mathbf{q}^\top \mathbf{B}(\Delta t)\mathbf{k}$ *grows* with $|\Delta t|$ instead of decaying (DAI *et al.*, 2025). Distant events would receive larger scores than nearby events, exactly the opposite of what we want.

Figure 8 illustrates this difference. In the left panel, cos and sin oscillate between -1 and 1 indefinitely, without any decay trend. In the right panel, cosh and sinh remove the oscillations but grow exponentially in both directions. Note that cosh is an even function (symmetric, with minimum value $\cosh(0) = 1$) while sinh is odd (antisymmetric, with $\sinh(0) = 0$).

4.3 Exponential decay and the parameter θ'

To correct the hyperbolic growth, the HoPE (DAI *et al.*, 2025) introduces a learnable parameter θ' and applies damping factors to the $\mathbf{q}$ and $\mathbf{k}$ vectors. The HoPE, like RoPE, was designed for language models, so m and n here are discrete token positions (the m-th and n-th

Figure 8 – Base functions of the attention kernels ($\theta = 1.0$). Left: trigonometric functions of the RoTHP, which oscillate between -1 and 1 without decaying. Right: hyperbolic functions of the HoPE, which are monotonic but grow exponentially. The growth of the hyperbolic functions needs to be compensated by a decay factor so that the attention kernel works correctly.



tokens in the sequence). The vector $\mathbf{q}$ at position m is multiplied by $e^{-m\theta'}$ and the vector $\mathbf{k}$ at position n is multiplied by $e^{+n\theta'}$. When computing the score between positions m and n, these factors combine:

$$g(\mathbf{x}_m, \mathbf{x}_n, n-m) = e^{-(m-n)\theta'}\, \mathbf{q}^\top \mathbf{B}(\theta, m-n)\, \mathbf{k}. \tag{4.3}$$

The contribution of this work is to adapt this mechanism, conceived for discrete token positions, to continuous time: in the HoTHP, we replace the discrete positions m, n by the continuous instants t_i, t_j and the difference $m-n$ by the temporal difference $\Delta t = t_i - t_j$:

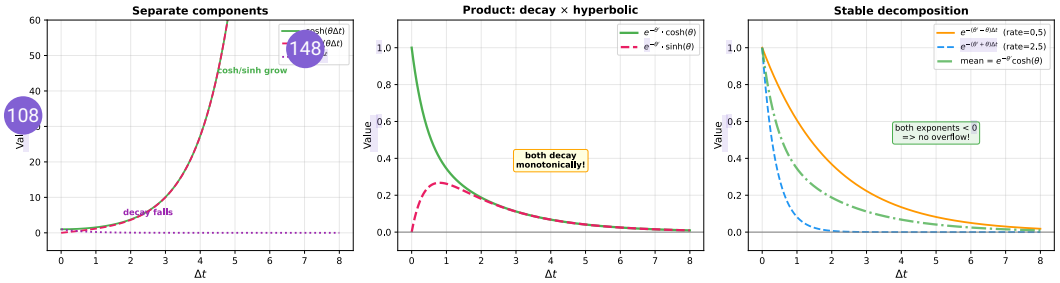
$$g(\Delta t) = e^{-|\Delta t|\,\theta'}\, \mathbf{q}^\top \mathbf{B}(\theta, \Delta t)\, \mathbf{k}. \tag{4.4}$$

For the decay to dominate the hyperbolic growth, it is necessary that $\theta' > \max_j \theta_j$. Under this condition, the score decreases monotonically with $|\Delta t|$, which introduces an exponential recency bias into the attention mechanism. This bias reflects the same structure as the Hawkes kernel ($\alpha e^{-\beta \Delta t}$) and, because it is structural and not learned from the data, it holds even for values of Δt that the model never saw during training. It is this bias that gives the HoTHP the ability to extrapolate to longer sequences without degradation.

Figure 9 illustrates the recency bias in practice. The left image shows the separate components: cosh and sinh grow exponentially while the decay factor $e^{-\theta'|\Delta t|}$ falls quickly to zero. The center image shows the product of the hyperbolic functions by the decay factor, ensuring that $\theta' > \theta$, the decay dominates the growth, and both $e^{-\theta'|\Delta t|}\cosh(\theta \Delta t)$ and $e^{-\theta'|\Delta t|}\sinh(\theta \Delta t)$ decay monotonically. The right image shows the decomposition into pure exponentials used in

the implementation (Section 4.5): since both exponents $(\theta' - \theta)$ and $(\theta' + \theta)$ are positive, all the exponentials have a negative argument and no overflow occurs.

Figure 9 – Effect of the exponential decay on the hyperbolic functions ($\theta = 1.0$, $\theta' = 1.5$). Left: separate components, showing that cosh and sinh grow but the decay falls. Center: the product results in functions that decay monotonically. Right: decomposition into pure exponentials, both with a negative exponent, ensuring numerical stability.



4.3.1 *Parametrization of the learnable θ'*

Since θ' needs to be learnable but at the same time satisfy $\theta' > \max_j \theta_j$, we cannot simply train a free parameter. The frequencies θ_j follow the RoPE formula (SU *et al.*, 2023):

$$\theta_j = 10000^{-2(j-1)/d}, \quad j = 1, \ldots, d/2, \tag{4.5}$$

where d is the dimension of each attention head. Each pair of dimensions receives a different frequency, covering multiple scales of Δt. Note that, by Equation (4.5), the largest value of θ_j occurs when $j = 1$, resulting in $\theta_1 = 10000^0 = 1$. All the other θ_j are increasingly smaller fractions (for example, $\theta_2 \approx 0.013$, $\theta_3 \approx 0.00017$, and so on). Therefore, in practice, the constraint $\theta' > \max_j \theta_j$ reduces to ensuring that $\theta' > 1$. Even so, we chose a general formulation that works regardless of the values of θ_j, so that the constraint is satisfied by construction. The solution is to parametrize θ' as:

$$\theta' = \text{softplus}(\theta'_{\text{raw}}) + \max_j(\theta_j) + \varepsilon, \tag{4.6}$$

where θ'_{raw} is the trainable parameter, $\text{softplus}(x) = \log(1 + e^x)$ ensures that the first term is positive, and $\varepsilon = 10^{-4}$ is a safety margin. Thus, θ' is always larger than any θ_j and the exponential decay is structurally guaranteed, regardless of what the gradient does.

4.4 Attention score of the HoTHP

Expanding the matrix multiplication of $\mathbf{B}_j$ (Equation (4.2)) over the even-dimension vectors $\mathbf{q}_1, \mathbf{k}_1$ and the odd-dimension vectors $\mathbf{q}_2, \mathbf{k}_2$, we obtain:

$$\mathbf{q}^\top \mathbf{B}_j(\Delta t)\mathbf{k} = \underbrace{(\mathbf{q}_1 \cdot \mathbf{k}_1 + \mathbf{q}_2 \cdot \mathbf{k}_2)}_{\text{cosh part}} \cdot \cosh(\theta_j \Delta t) + \underbrace{(\mathbf{q}_1 \cdot \mathbf{k}_2 + \mathbf{q}_2 \cdot \mathbf{k}_1)}_{\text{sinh part}} \cdot \sinh(\theta_j \Delta t). \tag{4.7}$$

The complete score incorporates the decay factor and sums over the $d/2$ pairs of dimensions:

$$s_{ij} = \frac{1}{\sqrt{d}} \sum_{j=1}^{d/2} \left[(q_1^{(j)} k_1^{(j)} + q_2^{(j)} k_2^{(j)}) \cdot e^{-|\Delta t|\theta'} \cosh(\theta_j \Delta t) + (q_1^{(j)} k_2^{(j)} + q_2^{(j)} k_1^{(j)}) \cdot e^{-|\Delta t|\theta'} \sinh(\theta_j \Delta t) \right], \tag{4.8}$$

where $\Delta t = t_i - t_j$ and $1/\sqrt{d}$ is the standard normalization of the *scaled dot-product attention* (VASWANI *et al.*, 2017).

In the RoTHP, the $\mathbf{q}$ and $\mathbf{k}$ vectors are pre-rotated individually by $\mathbf{R}(t_i)$ and $\mathbf{R}(t_j)$ before the inner product, and the dependence on Δt emerges from the algebraic cancellation demonstrated by the authors. In the HoTHP, the score is formulated directly in terms of Δt, which makes it natural and necessary to include the factor $e^{-|\Delta t|\theta'}$ inside the kernel itself. The practical consequence is that, for any Δt (including values larger than those seen in training), the score is dominated by the exponential decay. This is the mechanism that enables extrapolation: the recency bias does not depend on the data, it is in the structure of the attention.

4.5 Numerical stability

The functions cosh and sinh grow like $e^{|\Delta t|\theta_j}$ and, for events that are very far apart, these values can cause *overflow* even before being multiplied by the decay factor $e^{-|\Delta t|\theta'}$.

The solution is to distribute the decay into the hyperbolic functions using the identities:

$$e^{-|\Delta t|\theta'} \cdot \cosh(|\Delta t|\theta_j) = \frac{e^{-|\Delta t|(\theta' - \theta_j)} + e^{-|\Delta t|(\theta' + \theta_j)}}{2}, \tag{4.9}$$

$$e^{-|\Delta t|\theta'} \cdot \sinh(\Delta t \cdot \theta_j) = \operatorname{sgn}(\Delta t) \cdot \frac{e^{-|\Delta t|(\theta' - \theta_j)} - e^{-|\Delta t|(\theta' + \theta_j)}}{2}. \tag{4.10}$$

Since $\theta' > \max_j \theta_j$, we have $\theta' - \theta_j > 0$ and $\theta' + \theta_j > 0$, so both exponents are negative for any Δt and the *overflow* does not occur. The sign of Δt is reintroduced explicitly by $\operatorname{sgn}(\Delta t)$, since $\sinh(-x) = -\sinh(x)$.

4.6 Temporal normalization

Different datasets have very distinct temporal scales. In social networks, the intervals between events can be on the order of milliseconds, while in seismic data they are on the order of days. If we used the raw instants in the hyperbolic kernel, the values of $\theta_j \Delta t$ would be on incompatible scales and the model would need specific hyperparameters for each dataset.

We should be clear about where this normalization (the division by the mean inter-event interval defined below) is used. We apply it in the synthetic experiments and, among the real datasets, only in the Retweet ablation (Section 5.4), where controlling the temporal scale is part of the study. The other real datasets (Taxi, StackOverflow, and Amazon) are used with raw timestamps, only shifted so that each sequence starts at zero, without dividing by the mean. Their scales do not cause the numerical saturation that appears on Retweet, so the raw timestamps are kept in order to evaluate the models under the original conditions of each dataset.

The normalization applies two transformations to each sequence. First, we shift the instants so that the sequence starts at zero and, next, we divide all the instants by the mean of the inter-event intervals of that sequence:

$$\tilde{t}_i = \frac{t_i - t_1}{\bar{g}}, \tag{4.11}$$

where $\bar{g}$ is the arithmetic mean of the intervals between consecutive events of the sequence. This operation is done independently for each sequence. The shift ensures that $\tilde{t}_1 = 0$ and the division by $\bar{g}$ makes the mean interval between events become approximately 1 in all sequences, regardless of the original temporal scale of the dataset. The order and the proportion between the intervals are preserved.

The choice of this normalization was made after we evaluated two alternatives. The first was a MinMax normalization applied to the *intervals* between events, and not to the absolute instants: each $\Delta t_i = t_i - t_{i-1}$ is mapped to the interval $[0,1]$ by $\Delta\tilde{t}_i = (\Delta t_i - \Delta t_{\min})/(\Delta t_{\max} - \Delta t_{\min})$, where $\Delta t_{\min}$ and $\Delta t_{\max}$ are the smallest and the largest interval of the sequence. Applying the MinMax directly to the absolute instants would be inadequate, because the last event is always the maximum and, as the sequence grows, all the previous instants are pushed close to zero; normalizing the intervals avoids this problem. Even so, this alternative proved inadequate for the *train-short, test-long* protocol: in the synthetic extrapolation experiments it systematically favored the NHP, which does not depend on explicit positional embeddings, while the models with temporal attention (THP, RoTHP, and HoTHP) were harmed or showed instability, especially

at the high extrapolation factors.

The reason is that the MinMax divisor depends on the *maximum* interval of the sequence, and the maximum is an extreme value that tends to grow the more events the sequence has, since longer sequences have a greater chance of containing an atypically large interval. The mean normalization does not suffer from this problem because the divisor $\bar{g}$ is the mean of the intervals, a stable statistic that barely changes when more events from the same process are added: the normalized mean interval stays ≈ 1 at any length. An atypical interval is also diluted by the mean, whereas in the MinMax it would change the maximum and compress all the others.

The second alternative evaluated was a temporal scale factor learned as a parameter of the model during training. This approach introduced instability in the optimization, with high variance between seeds at the high extrapolation factors. The mean-interval normalization, because it is simple and does not depend on the length of the test sequence, was adopted in the experiments with synthetic data and in the Retweet ablation (Section 5.4).

4.7 Complete architecture

The HoTHP architecture follows the Transformer encoder (VASWANI *et al.*, 2017). Each event (t_i, k_i) is a pair of a time t_i and a type k_i (the category, or mark, of the event). The type k_i is represented by a learnable embedding $\mathbf{e}_{k_i} \in \mathbb{R}^d$. Unlike the THP (ZUO *et al.*, 2021), which adds a sinusoidal encoding of the absolute time to the embedding, the HoTHP does not include any temporal information in the embedding. The instants t_i enter the model only through the hyperbolic kernel.

The model stacks N hyperbolic encoder layers, each one with a hyperbolic multi-head attention block followed by a *feed-forward* network. The attention computes the scores via Equation (4.8), using the normalized differences $\Delta\tilde{t}_{ij} = \tilde{t}_i - \tilde{t}_j$. A causal mask ensures that each event attends only to previous events.

The *feed-forward* network in each layer has two linear projections with a ReLU between them:

$$\mathrm{FFN}(\mathbf{x}) = \mathbf{W}_2 \cdot \mathrm{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2, \tag{4.12}$$

with internal dimension $2d$. Like the RoTHP (DAI *et al.*, 2026), the HoTHP does not use residual connections (add & norm).

The encoder output $\mathbf{h}(t_j) \in \mathbb{R}^d$ for event j is used to compute the conditional intensity

of each type k at the instant $t > t_j$:

$$\lambda_k(t \mid \mathcal{H}_t) = \text{softplus}\left(\gamma_k\,(t - t_j) + \mathbf{w}_k^\top \mathbf{h}(t_j) + b_k\right), \tag{4.13}$$

where γ_k, $\mathbf{w}_k$, and b_k are learnable parameters. Each term has a clear role. The baseline b_k is the level that remains for type k when there is no contribution from the history or from the elapsed time, analogous to the μ of the classical Hawkes process (Equation (2.20)). The history term $\mathbf{w}_k^\top \mathbf{h}(t_j)$ reads the hidden state $\mathbf{h}(t_j)$ that the hyperbolic attention built from all the past events, and $\mathbf{w}_k$ projects it onto type k. The current term $\gamma_k(t - t_j)$ controls how the intensity changes with the time elapsed since the last event t_j, with γ_k a learnable scalar for each type. Finally, softplus keeps the intensity positive. Following the implementation in the EasyTPP framework (XUE *et al.*, 2024), this current term is linear in $t - t_j$; the original THP (ZUO *et al.*, 2021) instead normalizes it by t_j (Equation (3.4)), a division we do not use here.

4.8 Where the exponential bias acts

An important distinction needs to be made about the role of the exponential decay in the HoTHP. The factor $e^{-|\Delta t|\theta'}$ does not act directly on the intensity function $\lambda_k(t)$, but on the attention mechanism that produces the hidden state $\mathbf{h}(t_j)$. These are two distinct steps of the model, with different roles:

1. **Hyperbolic attention** (Equation (4.8)): determines the weights with which each past event contributes to the hidden state. The exponential decay makes recent events receive larger weights and distant events receive weights close to zero. The result is the vector $\mathbf{h}(t_j)$, which encodes the historical context of the sequence.
2. **Intensity function** (Equation (4.13)): uses the hidden state $\mathbf{h}(t_j)$ and the time elapsed since the last event to compute $\lambda_k(t)$. The sign of the learnable parameter γ_k decides whether type k is self-exciting or self-inhibiting, as explained just after Equation (4.13). What matters here is that this decision is made in the intensity layer, and not by the exponential decay of the attention.

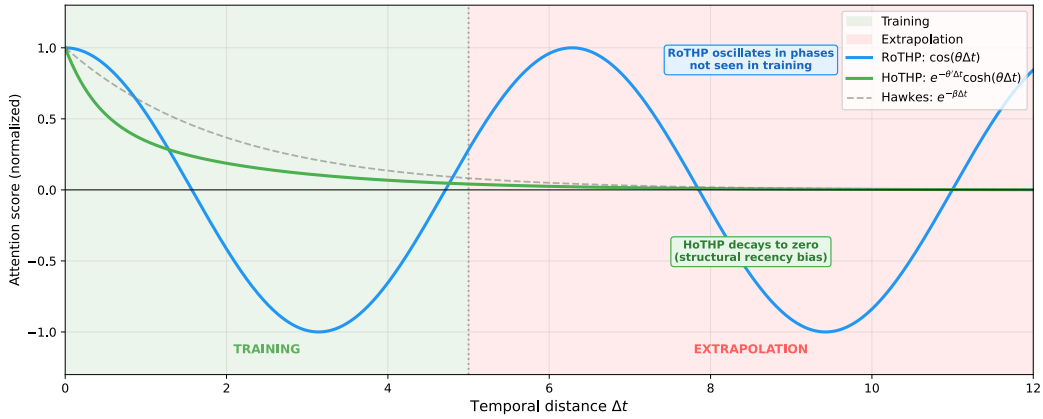
This separation has an important consequence, which is that the exponential bias of the HoTHP does not require the modeled process to be self-exciting. The hyperbolic kernel only requires that the relevance of past events for the construction of the hidden state decay exponentially with the temporal distance. The dynamics of the intensity itself (whether it rises or falls after an event) is determined by the learnable parameters of the intensity layer, which are

free to adapt to the data.

Therefore, the HoTHP is suitable for any point process in which the most recent event carries more information than distant events for predicting the next one. This includes both self-exciting processes (where events generate more nearby events) and self-inhibiting ones (where the occurrence of an event temporarily reduces the probability of new events). The bias is harmful when distant events are as relevant as, or more relevant than, recent ones, as in processes without temporal memory or with long-range dependencies.

Figure 10 visually summarizes the difference between the attention kernels of the RoTHP and the HoTHP. In the training zone (green background), both models observe the same temporal distances and can learn appropriate attention weights. In the extrapolation zone (red background), the trigonometric kernel of the RoTHP keeps oscillating in phases that the model never saw during training, assigning potentially high scores to very distant events. The hyperbolic kernel of the HoTHP, on the other hand, decays monotonically to zero regardless of the distance, thanks to the structural bias. For reference, the dashed curve shows the classical Hawkes kernel ($e^{-\beta\Delta t}$), whose shape the HoTHP reproduces by its own architecture. The figure also makes clear that the HoTHP suppresses distant events more sharply than the RoTHP. This sharp, monotonic decay is what stabilizes extrapolation. It is also why the recency bias brings little benefit when the process has genuinely long-range dependencies, as discussed above.

Figure 10 – Comparison of the attention kernels as a function of the temporal distance Δt. In the training zone, both models can fit the data. In the extrapolation zone, the trigonometric kernel of the RoTHP oscillates in phases that were not learned, while the hyperbolic kernel of the HoTHP decays monotonically, like the classical Hawkes kernel.



4.9 Experimental setup

The experiments use two types of data: synthetic and real. The synthetic data is generated by Hawkes processes with an exponential kernel. The choice of synthetic data is deliberate: since we know the generating process, we can check whether the recency bias of the HoTHP actually aligns with the real decay of the data and whether this correspondence is what sustains the extrapolation. The real data are the datasets provided by the EasyTPP library (XUE *et al.*, 2024): Taxi, StackOverflow, Retweet, and Amazon. These datasets cover varied domains (transportation, question forums, social networks, and electronic commerce) and allow us to evaluate the performance of the HoTHP in scenarios where the generating process is unknown.

4.9.1 Synthetic data generation

We generate sequences from multivariate Hawkes processes with $K = 2$ event types and an exponential kernel. We define two generation scenarios to cover distinct regimes of temporal dependence.

In the **slow decay scenario**, the parameter β is small, so that the influence of past events persists for longer and distant events still affect the present intensity.

In the **fast decay scenario**, β is large and only recent events have relevant influence.

For each scenario, we generate 400 training sequences, 100 validation sequences, and 200 test sequences. The validation sequences have the same length as the training ones and are used only for early stopping, so model selection never sees the extrapolation regime. Only the test sequences are extended to the longer lengths. All the sequences are preprocessed with the temporal normalization of Equation (4.11) before being fed to the models.

4.9.2 Train short, test long protocol

The *train short, test long* protocol, proposed by Press *et al.* (2021) for language models and already discussed in Section 3.3, trains the model on short sequences and tests it on longer sequences. We adapted the protocol to point processes using the number of events as the measure of length.

We fix the training length at $L = 50$ events and vary the extrapolation factor ρ over $\{1\times, 2\times, 5\times, 10\times\}$, where ρ indicates how many times longer the test sequence is than the training one. With $\rho = 10\times$, for example, we train with sequences of 50 events and test with

sequences of 500. Each factor is evaluated in the two decay scenarios.

4.9.3 Compared models and implementation

On the synthetic data we compare four models: NHP (MEI; EISNER, 2017), THP (ZUO *et al.*, 2021), RoTHP (DAI *et al.*, 2026), and the proposed HoTHP. The RoTHP is the closest baseline, since it shares exactly the same encoder structure and the same intensity function as the HoTHP, differing only in the positional kernel; the NHP (recurrent) and the THP (absolute temporal encoding) serve to situate the extrapolation behavior with respect to distinct architectures. On the real data we compare the four models (NHP, THP, RoTHP, and HoTHP) on Taxi, Amazon, and Stackoverflow, and the three Transformer models (THP, RoTHP, and HoTHP) on Retweet. All four models compute the NLL with the same likelihood routine (the same Monte Carlo estimate of the compensator integral, defined once in the base class), so the comparison of likelihoods does not depend on the way each model approximates the integral.

All the experiments were built on top of EasyTPP (XUE *et al.*, 2024), an open benchmark for neural temporal point processes that provides, in a single framework, standardized datasets, reference implementations of the main neural TPP models, and a common training and evaluation pipeline. From EasyTPP we used the four real datasets, the reference implementations of the NHP and the THP, and the shared training and evaluation code. The RoTHP and the HoTHP were implemented as subclasses of the same THP base class, changing only the positional kernel, so that all the models share the same encoder, the same intensity layer, and the same evaluation routine.

The baselines share the same hyperparameters as the HoTHP in each scenario, so as to isolate the effect of the positional kernel. In the synthetic experiments we use a model dimension $d = 32$, a temporal *embedding* dimension equal to 32, $N_\ell = 2$ layers, $H = 2$ attention heads, *dropout* of 0.1, and the AdamW optimizer (*weight decay* 10^{-4}) with learning rate 10^{-3}, except the HoTHP, which uses 5×10^{-4} for greater numerical stability of the hyperbolic functions. In the experiments with real data we use $d = 64$, a temporal *embedding* equal to 16, $N_\ell = 2$ layers, $H = 2$ heads, and a learning rate of 10^{-3} for all the models. The architecture hyperparameters (number of layers, attention heads, and embedding sizes) follow the reference configurations of the EasyTPP framework (XUE *et al.*, 2024). Only the learning rates were tuned in this work, as justified below.

The two learning rates deserve a brief justification. On the synthetic data, the

hyperbolic kernel of the HoTHP trained less stably at 10^{-3}, so we used 5×10^{-4} for it, while the baselines were stable at 10^{-3}. On the real data, a grid search for the HoTHP on Amazon over $\{10^{-3}, 5 \times 10^{-4}, 10^{-4}, 5 \times 10^{-5}\}$ (3 seeds) selected 10^{-3}, which gave the lowest NLL at every factor and degraded monotonically for smaller rates; we adopted it for all the models to keep the comparison fair. In summary, the HoTHP uses 5×10^{-4} on the synthetic data and 10^{-3} on the real data, while all the other models use 10^{-3} throughout.

4.9.4 Metrics and statistical evaluation

The evaluation metric is the NLL, computed over the complete test sequence, including the events beyond the training length. If the recency bias of the HoTHP indeed enables extrapolation, the NLL should remain stable even when the test sequence is much longer than the training one. We also report RMSE for the predicted time of the next event and Accuracy for its predicted type. Both are computed one step ahead with teacher forcing: at each position of the sequence, from the true history up to that point, the model predicts the next event, and each prediction is compared with the real next event. There is therefore one prediction per transition between consecutive events, aligned one-to-one with the observed sequence. A mismatch in the number or timing of events would only arise under free generation, which is not done for these metrics.

We repeat the synthetic experiments with $N = 10$ seeds and the experiments with real data with $N = 3$ seeds, reporting mean $\pm$ standard deviation. To check statistical significance in the synthetic experiments, we use the *Wilcoxon signed-rank test*, a non-parametric paired test that does not assume normality and is suitable for small samples. The pairing is by seed: each seed generates its own draw of the synthetic data, so different seeds produce different datasets, and the same seed is reused across all the models, so the same seed produces the same training, validation, and test set for every model. With $N = 10$, the smallest attainable one-sided p-value is $1/2^{10} \approx 0.001$, sufficient for the level $\alpha = 0.05$.

4.9.5 Computational environment

All the experiments were run on a machine with an NVIDIA A100-SXM4-80GB GPU (80 GB of VRAM), 6 physical cores (12 threads), 167 GB of RAM, and CUDA 12.8. The code uses PyTorch 2.10.0 and Python 3.12.

5 RESULTS

This chapter presents the experimental results of the HoTHP in two contexts: synthetic data, where the generating process is known, and real data, where it is not. Next, we visualize the intensities learned by the models to understand the behavior of the recency bias in practice, and we run an ablation to isolate the effect of the temporal scale.

5.1 Synthetic data

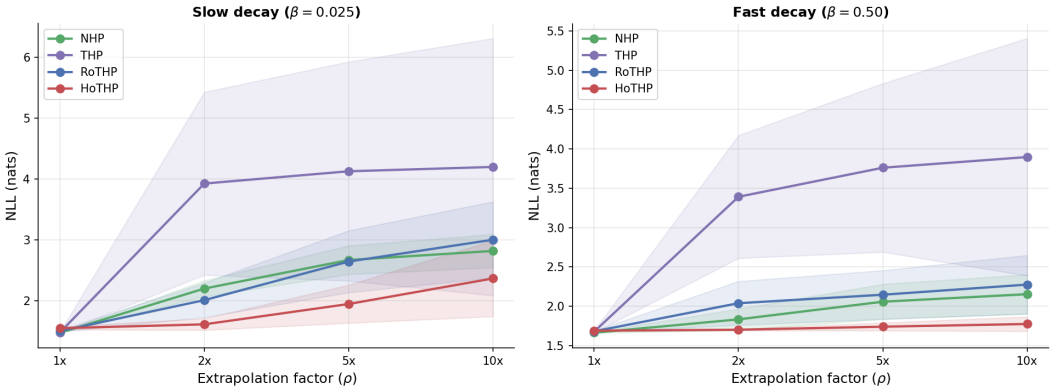
The synthetic experiments follow the protocol described in Section 4.9, using Hawkes processes with an exponential kernel in the slow decay (β small) and fast decay (β large) scenarios. We compare four models: NHP, THP, RoTHP, and HoTHP. Table 5 summarizes the results for the *train short, test long* protocol, varying the extrapolation factor ρ from $1\times$ (training length) to $10\times$. Figure 11 visualizes the evolution of the NLL as a function of ρ.

Table 5 – NLL (mean $\pm$ standard deviation, 10 seeds) on the synthetic data. Lower values indicate better performance. The best result for each configuration is in bold and the second best is underlined.

Scenario	Model	$1\times$	$2\times$	$5\times$	$10\times$
Slow decay	NHP	$\mathbf{1.470 \pm 0.007}$	2.197 ± 0.131	2.667 ± 0.236	$\underline{2.817 \pm 0.273}$
	THP	$\underline{1.475 \pm 0.007}$	3.926 ± 1.505	4.127 ± 1.803	4.199 ± 2.117
	RoTHP	1.499 ± 0.014	$\underline{2.007 \pm 0.284}$	$\underline{2.642 \pm 0.509}$	3.001 ± 0.625
	HoTHP	1.545 ± 0.025	$\mathbf{1.609 \pm 0.096}$	$\mathbf{1.943 \pm 0.313}$	$\mathbf{2.364 \pm 0.625}$
Fast decay	NHP	$\mathbf{1.663 \pm 0.005}$	$\underline{1.833 \pm 0.134}$	$\underline{2.057 \pm 0.223}$	$\underline{2.153 \pm 0.250}$
	THP	$\underline{1.677 \pm 0.011}$	3.391 ± 0.781	3.761 ± 1.072	3.897 ± 1.509
	RoTHP	1.681 ± 0.007	2.039 ± 0.279	2.146 ± 0.309	2.275 ± 0.372
	HoTHP	1.689 ± 0.008	$\mathbf{1.701 \pm 0.012}$	$\mathbf{1.740 \pm 0.050}$	$\mathbf{1.774 \pm 0.092}$

With the inclusion of the NHP and the THP as baselines, four distinct behaviors emerge in the extrapolation. The THP is the model that degrades the most: its NLL jumps to ≈ 3.9 already at $2\times$ and stays between ≈ 3.9 and ≈ 4.2 at the larger factors, in both scenarios, with a very high standard deviation (up to ± 2.12) caused by seeds that diverge completely. This confirms that the absolute temporal encoding of the THP produces representations outside the distribution for new positions. The NHP, despite not using attention, is the most competitive baseline in the extrapolation, probably because its recurrent architecture accumulates information incrementally, without depending on absolute positions. Even so, it stays behind the HoTHP at all extrapolation factors in both scenarios.

Figure 11 – NLL (mean ± standard deviation, 10 seeds) as a function of the extrapolation factor ρ in the synthetic scenarios. The HoTHP keeps the NLL more stable as ρ grows, while the THP degrades drastically and the RoTHP and NHP show intermediate degradation.



The HoTHP obtains the best NLL at all extrapolation factors (2×, 5×, and 10×) in both scenarios. The cost of this advantage appears in distribution: at 1×, the HoTHP is the model with the worst NLL (for example, 1.545 against 1.470 of the NHP in the slow decay), a small difference that reverses as soon as the test sequence goes beyond the training length. In the fast decay, the NLL of the HoTHP grows by only 0.085 between 1× and 10× (1.689 → 1.774), staying practically flat, while the second best model (NHP) grows by 0.490 (1.663 → 2.153). This stability is expected: since the generating process has an exponential kernel, the recency bias of the HoTHP matches the real structure of the data. For temporal distances not seen in training, the hyperbolic kernel keeps decaying monotonically, while the trigonometric kernel of the RoTHP oscillates in phases that were not learned and the absolute encoding of the THP diverges.

The advantage of the HoTHP over the three baselines is statistically significant (Wilcoxon signed-rank test, paired by seed and one-sided, $p < 0.05$) at all extrapolation factors of both scenarios, without exception. Table 6 shows all the p-values.

In the slow decay, where the influence of distant events persists for longer, the HoTHP still obtains the best mean NLL at all factors, but its degradation is much larger than in the fast decay: the NLL grows by 0.819 between 1× and 10×, against only 0.085 in the fast decay. At 10×, the mean of the HoTHP (2.364) is lower than that of the NHP (2.817) and the difference remains statistically significant, even if by the narrowest margin of the whole experiment ($p = 0.042$), reflecting the high variance between seeds in this regime (±0.625). This suggests that, when the memory of the process is long, the exponential decay imposed by the hyperbolic kernel may discard events that are still relevant too early, reducing the advantage

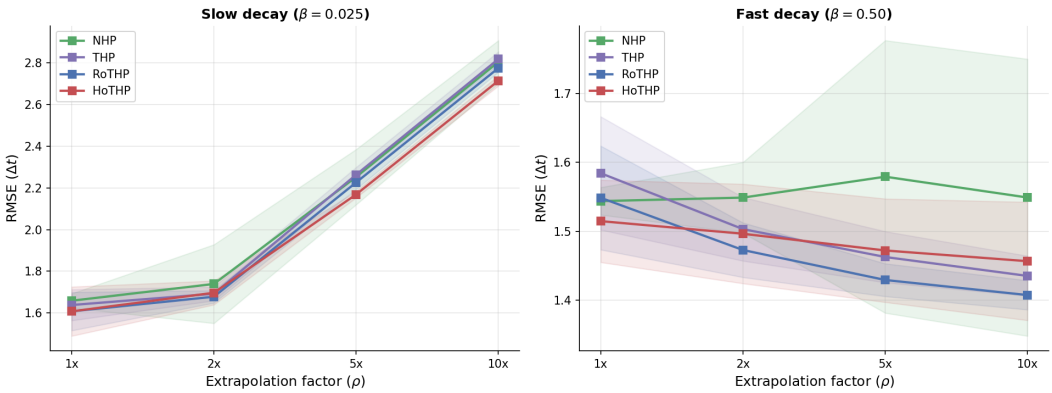
Table 6 – p-values of the Wilcoxon test comparing the HoTHP with each baseline on the synthetic data (one-sided, paired by seed, 10 seeds). Δ is the mean NLL of the HoTHP minus the baseline, so a negative value means the HoTHP is better.

		vs NHP		vs THP		vs RoTHP	
Scenario	ρ	Δ	p	Δ	p	Δ	p
Slow decay	1×	+0.074	1.000	+0.069	1.000	+0.046	1.000
	2×	−0.589	0.001	−2.317	0.001	−0.398	0.001
	5×	−0.724	0.001	−2.184	0.001	−0.698	0.005
	10×	−0.453	0.042	−1.835	0.005	−0.637	0.024
Fast decay	1×	+0.026	1.000	+0.013	0.990	+0.008	0.995
	2×	−0.132	0.002	−1.690	0.001	−0.337	0.001
	5×	−0.318	0.001	−2.021	0.001	−0.406	0.001
	10×	−0.379	0.001	−2.122	0.001	−0.501	0.001

margin.

Figure 12 shows the RMSE of the next event prediction. In the slow decay, all the models degrade in a similar way (RMSE $\approx$ 2.7–2.8 at 10×), with the HoTHP showing the lowest RMSE (2.714). In the fast decay, the four models keep a stable and close RMSE (between $\approx$ 1.40 and 1.55 at 10×): the RoTHP obtains the lowest value (1.407) and the NHP the highest (1.549), the latter with a slightly higher variance due to an atypical seed. Unlike the NLL, the RMSE does not separate the models clearly. This indicates that the advantage of the HoTHP shows up mainly in the likelihood, that is, in modeling the full temporal distribution, and not in the pointwise prediction of the next interval, where the attention models are practically tied.

Figure 12 – RMSE of the next event prediction (mean $\pm$ standard deviation, 10 seeds) as a function of the extrapolation factor ρ. Unlike the NLL, the RMSE does not separate the models clearly: in the slow decay all of them degrade in a similar way and in the fast decay they stay close to each other.



5.2 Real data

To evaluate the HoTHP in more realistic scenarios, we ran experiments on four public point process datasets. Table 7 describes each dataset.

Table 7 – Real datasets used in the experiments. L_{train} is the maximum training length (number of events) and $L_{5\times}$ is the test length in the extrapolation condition.

Dataset	Types	Sequences (train)	L_{train}	$L_{5\times}$	Domain
Amazon	16	6,454	18	90	Product reviews
Stackoverflow	22	1,401	20	100	User badges
Taxi	10	1,400	7	38	Taxi rides
Retweet	3	20,000	50	250	Retweet cascades

In all four datasets, each sequence is a marked temporal point process: every event has a timestamp and a discrete type (the mark). The model has two jobs, to predict when the next event happens (the time to the next event) and which type it will be. What changes from one dataset to another is what an event represents and what the types mean. The descriptions below follow the benchmark definitions of (XUE *et al.*, 2024).

In Amazon, each sequence is the stream of product reviews written by one user, ordered in time. An event is a review, and its type is the category of the reviewed product (16 categories). In Stackoverflow, each sequence is the history of badges earned by one user on the question-and-answer website over about two years. An event is a badge award, and its type is which of the 22 badges was received. In Taxi, each sequence is a stream of taxi pick-up and drop-off events across the five boroughs of New York City. An event is a pick-up or a drop-off, and each combination of borough and kind of event (pick-up or drop-off) is a type, giving 10 types. In Retweet, each sequence is a retweet cascade that starts from an original tweet. An event is a retweet, and its type is the group of the user who retweeted, split by number of followers into three classes (small, fewer than 120 followers; medium, fewer than 1363; and large, the rest).

We follow the same *train short, test long* protocol: we train each model with sequences truncated at L_{train} events and evaluate at the training length ($1\times$) and at two extrapolation factors ($2\times$ and $5\times$). We use three seeds (42, 142, 242), 100 epochs, and the same hyperparameters as in Section 4.9.3. On Taxi, Amazon, and Stackoverflow we compare the four models (THP, RoTHP, HoTHP, and NHP); on Retweet we compare the three Transformer models (THP, RoTHP, and HoTHP), since this dataset is used mainly to study numerical stability (Section 5.4).

As explained in Section 4.9.3, all four models compute the NLL with the same

likelihood routine, so the differences reported below come from the models and not from the way each one approximates the compensator integral. With only three seeds we report the mean and the standard deviation, but we do not apply the Wilcoxon test on the real data. With $N = 3$ the smallest attainable one-sided p-value is $1/2^3 = 0.125$, above the level $\alpha = 0.05$, so we use the test only in the synthetic experiments of the previous section (Section 5.1), which use $N = 10$ seeds. Table 8 presents the NLL and Table 9 the type-prediction accuracy.

Table 8 – NLL on the real datasets (mean ± standard deviation, 3 seeds), for the extrapolation factors 1×, 2×, and 5×. Lower is better. The best value in each column is in bold and the second best is underlined. NHP is a recurrent baseline (CT-LSTM); the other three share the same Transformer architecture and intensity function and differ only in the positional kernel.

Dataset	Model	NLL 1×	NLL 2×	NLL 5×
Taxi	THP	-0.273 ± 0.002	-0.314 ± 0.004	-0.243 ± 0.015
	RoTHP	-0.271 ± 0.003	-0.317 ± 0.005	-0.137 ± 0.148
	HoTHP	$\underline{-0.278 \pm 0.001}$	$\underline{-0.328 \pm 0.004}$	$\underline{-0.266 \pm 0.007}$
	NHP	$\mathbf{-0.449 \pm 0.004}$	$\mathbf{-0.506 \pm 0.004}$	$\mathbf{-0.460 \pm 0.003}$
Amazon	THP	2.264 ± 0.003	2.237 ± 0.002	2.254 ± 0.002
	RoTHP	2.268 ± 0.004	2.230 ± 0.003	2.247 ± 0.002
	HoTHP	$\underline{2.263 \pm 0.004}$	$\underline{2.207 \pm 0.003}$	$\underline{2.187 \pm 0.002}$
	NHP	$\mathbf{0.658 \pm 0.229}$	$\mathbf{0.630 \pm 0.227}$	$\mathbf{0.638 \pm 0.224}$
Stackoverflow	THP	$\underline{2.744 \pm 0.029}$	$\underline{2.642 \pm 0.025}$	$\mathbf{2.519 \pm 0.013}$
	RoTHP	2.766 ± 0.035	2.655 ± 0.026	2.545 ± 0.018
	HoTHP	2.758 ± 0.025	2.649 ± 0.022	2.536 ± 0.021
	NHP	$\mathbf{2.742 \pm 0.012}$	$\mathbf{2.633 \pm 0.013}$	$\underline{2.524 \pm 0.020}$

Two layers of comparison appear in these tables. The first is the comparison *within the Transformer family* (THP, RoTHP, and HoTHP), which share the same architecture and the same intensity function and differ only in the positional kernel. This is the controlled comparison that isolates the effect of our contribution. The second is the comparison against the NHP, a recurrent model of a different family, included to situate the behavior across architectures.

Within the Transformer family. On Amazon, the HoTHP obtains the best NLL among the three at every factor and, more importantly, it is the only one that keeps improving as the sequence grows: its NLL drops monotonically from 2.263 at 1× to 2.207 at 2× and 2.187 at 5×, while the THP and the RoTHP improve at 2× but worsen again at 5×, returning to about 2.25. The type accuracy follows the same trend, rising from 0.312 to 0.340, the largest value among all the models. On Taxi, the HoTHP again has the best NLL of the three at every factor and is the most stable in extrapolation: at 5× the RoTHP degrades to -0.137 with a very

Table 9 – Type-prediction accuracy on the real datasets (mean ± standard deviation, 3 seeds), for the factors 1×, 2×, and 5×. Higher is better. The best value in each column is in bold and the second best is underlined.

Dataset	Model	Acc 1×	Acc 2×	Acc 5×
Taxi	THP	**0.900 ± 0.001**	0.917 ± 0.003	0.911 ± 0.002
	RoTHP	0.898 ± 0.000	0.916 ± 0.002	0.907 ± 0.011
	HoTHP	0.897 ± 0.001	0.917 ± 0.003	0.915 ± 0.004
	NHP	0.898 ± 0.002	**0.919 ± 0.001**	**0.918 ± 0.001**
Amazon	THP	0.308 ± 0.001	0.317 ± 0.003	0.314 ± 0.006
	RoTHP	0.308 ± 0.002	0.323 ± 0.002	0.320 ± 0.007
	HoTHP	**0.312 ± 0.001**	**0.331 ± 0.001**	**0.340 ± 0.001**
	NHP	0.293 ± 0.004	0.306 ± 0.006	0.315 ± 0.006
Stackoverflow	THP	0.473 ± 0.001	0.466 ± 0.001	0.450 ± 0.001
	RoTHP	0.473 ± 0.004	0.464 ± 0.002	0.448 ± 0.001
	HoTHP	**0.474 ± 0.003**	**0.466 ± 0.002**	**0.451 ± 0.001**
	NHP	0.469 ± 0.003	0.462 ± 0.002	0.445 ± 0.002

high standard deviation (±0.148, a sign that some seeds diverge), while the HoTHP stays at -0.266 ± 0.007. On Stackoverflow the three models are practically tied (differences within the error margin), with the HoTHP slightly ahead in accuracy at every factor; this dataset does not seem to have a temporal structure that separates the positional biases.

Against the NHP. The NHP obtains the lowest NLL on Taxi and Amazon (on Amazon by a large margin, ≈ 0.64 against ≈ 2.2) and is tied with the THP on Stackoverflow. This result is expected and does not contradict our contribution, for two reasons. First, it agrees with the unified benchmark of the area: when all the models are reimplemented with the same likelihood routine, the recurrent NHP is the strongest on several datasets, winning exactly on Taxi and Retweet in the EasyTPP benchmark (XUE *et al.*, 2024), because its continuous-time formulation models the density between events natively. Second, the NLL is only one of the metrics: on type accuracy the HoTHP surpasses the NHP on Amazon (0.340 against 0.315) and on Stackoverflow (0.451 against 0.445), so the advantage of the NHP is restricted to the likelihood. The research question of this work is not whether attention beats recurrence in absolute NLL, but whether changing the positional kernel improves the extrapolation of a Transformer. For that question the relevant comparison is the controlled one above, in which the HoTHP is consistently the best of the three.

Retweet. This dataset is treated separately in Section 5.4, because with raw timestamps the NLL of the three Transformer models overflows numerically (the extreme temporal scale, with intervals of up to 582,928 time units, makes the hyperbolic and trigonometric kernels

saturate, as discussed in Section 4.5). The ablation in Section 5.4 normalizes the scale and isolates the effect of the kernel.

Figure 13 shows the NLL as a function of the extrapolation factor. We omit the NHP from the plot: its much lower NLL on Taxi and Amazon compresses the vertical scale and hides the differences among the Transformer models, which are the focus of the controlled comparison. The NHP values stay in Table 8. The figure makes the within-family behavior clear: the HoTHP (red) keeps the lowest NLL among the three and the most stable curve, while the RoTHP (blue) shows the widest error band on Taxi at 5× and the only clear divergence on the normalized Retweet.

Figure 13 – NLL (mean ± standard deviation, 3 seeds) as a function of the extrapolation factor on the real datasets, for the three Transformer models. The NHP is omitted from the plot for readability, since its lower NLL on Taxi and Amazon would compress the scale; its values are in Table 8. Among the Transformer models, the HoTHP keeps the lowest and most stable NLL in extrapolation.

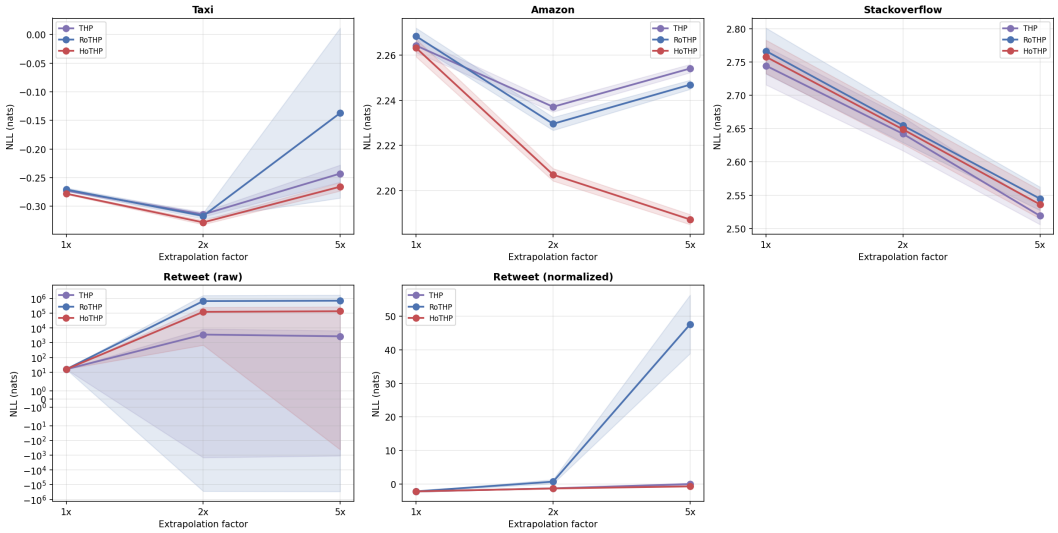
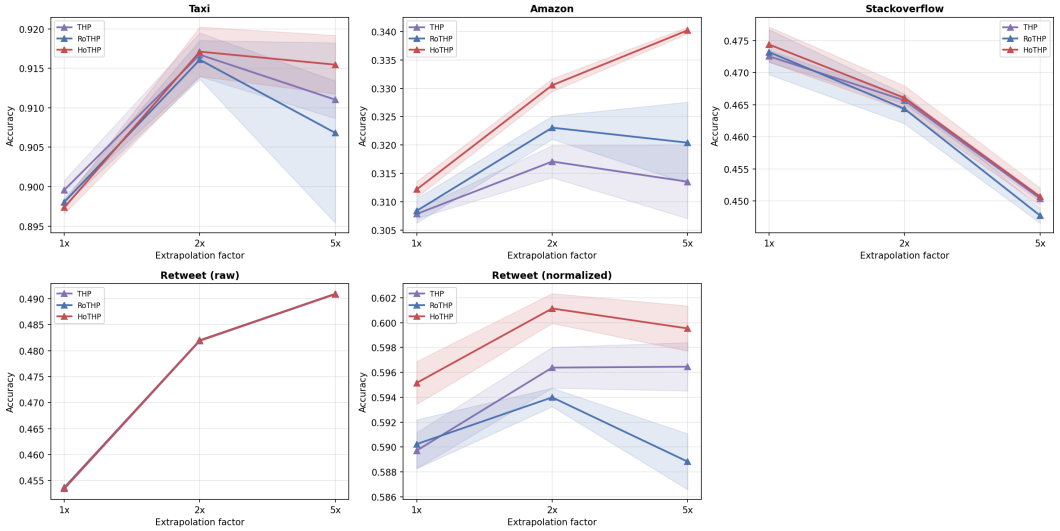


Figure 14 shows the type-prediction accuracy under the same protocol. On Amazon the recency bias of the HoTHP produces a clear and growing advantage: its accuracy rises to 0.340 at 5×, well above the THP and the RoTHP, which stay around 0.31–0.32. On Stackoverflow the HoTHP stays slightly ahead at every factor, and on Taxi the three models are close. As in the NLL plot, the NHP is omitted from the figure to keep the comparison clean within the Transformer family; its accuracy is in Table 9, where the HoTHP also surpasses the NHP on Amazon and Stackoverflow.

Table 10 completes the picture with the RMSE of the next-event time prediction. Unlike the NLL, the RMSE stays close across the models. On Amazon the four models are

Figure 14 – Type-prediction accuracy (mean ± standard deviation, 3 seeds) as a function of the extrapolation factor on the real datasets, for the three Transformer models (the NHP is in Table 9). On Amazon, the HoTHP shows the clearest and growing advantage as the sequence gets longer.



essentially tied around 0.48, and the value barely changes with the extrapolation factor. On Stackoverflow the RMSE actually decreases as the sequences get longer (from about 1.8 at 1× to about 1.46 at 5×). On Taxi the NHP has the lowest RMSE at every factor, and among the Transformer models the HoTHP is the best at 2× and 5×. As on the synthetic data, the RMSE does not separate the models the way the NLL does, and no positional kernel is consistently ahead across the datasets. On the real data, then, the strongest result for the HoTHP is the type-prediction accuracy, not the RMSE or the NLL (where the recurrent NHP is the hardest baseline to beat). The HoTHP is the best model on Amazon and Stackoverflow (Table 9), ahead even of the NHP, and within the Transformer family it stays the most consistent of the three across the metrics.

5.2.1 Training convergence

Figure 15 shows the convergence curves of the validation NLL over the 100 training epochs. On the Amazon and Stackoverflow datasets, the three models converge smoothly, with the NLL stabilizing after epoch 50. On Amazon, the curve still shows a slight downward trend in the last 20 epochs, but with a slope below 0.01 per epoch, which indicates that additional epochs would benefit all the models equally without changing the ranking.

On Taxi, the three models end up at the same level (NLL ≈ -0.24), but they get there in different ways: the RoTHP and the HoTHP reach it around epoch 20, while the THP

Table 10 – RMSE of the next-event time prediction on the real datasets (mean ± standard deviation, 3 seeds), for the extrapolation factors 1×, 2×, and 5×. Lower is better. The best value in each column is in bold and the second best is underlined.

Dataset	Model	RMSE 1×	RMSE 2×	RMSE 5×
Taxi	THP	0.419 ± 0.027	0.452 ± 0.055	0.452 ± 0.043
	RoTHP	$\underline{0.399 \pm 0.019}$	0.405 ± 0.014	0.457 ± 0.052
	HoTHP	0.414 ± 0.027	$\underline{0.402 \pm 0.030}$	$\underline{0.431 \pm 0.040}$
	NHP	$\mathbf{0.374 \pm 0.004}$	$\mathbf{0.380 \pm 0.013}$	$\mathbf{0.426 \pm 0.013}$
Amazon	THP	0.487 ± 0.005	0.488 ± 0.006	0.485 ± 0.005
	RoTHP	$\underline{0.486 \pm 0.002}$	$\underline{0.484 \pm 0.001}$	$\underline{0.480 \pm 0.001}$
	HoTHP	0.489 ± 0.001	0.487 ± 0.001	0.484 ± 0.001
	NHP	$\mathbf{0.486 \pm 0.011}$	$\mathbf{0.482 \pm 0.013}$	$\mathbf{0.474 \pm 0.014}$
Stackoverflow	THP	1.799 ± 0.026	$\underline{1.586 \pm 0.030}$	$\mathbf{1.428 \pm 0.039}$
	RoTHP	1.808 ± 0.017	1.614 ± 0.028	1.481 ± 0.041
	HoTHP	$\underline{1.783 \pm 0.010}$	1.593 ± 0.012	1.463 ± 0.009
	NHP	$\mathbf{1.772 \pm 0.025}$	$\mathbf{1.580 \pm 0.025}$	$\underline{1.460 \pm 0.035}$

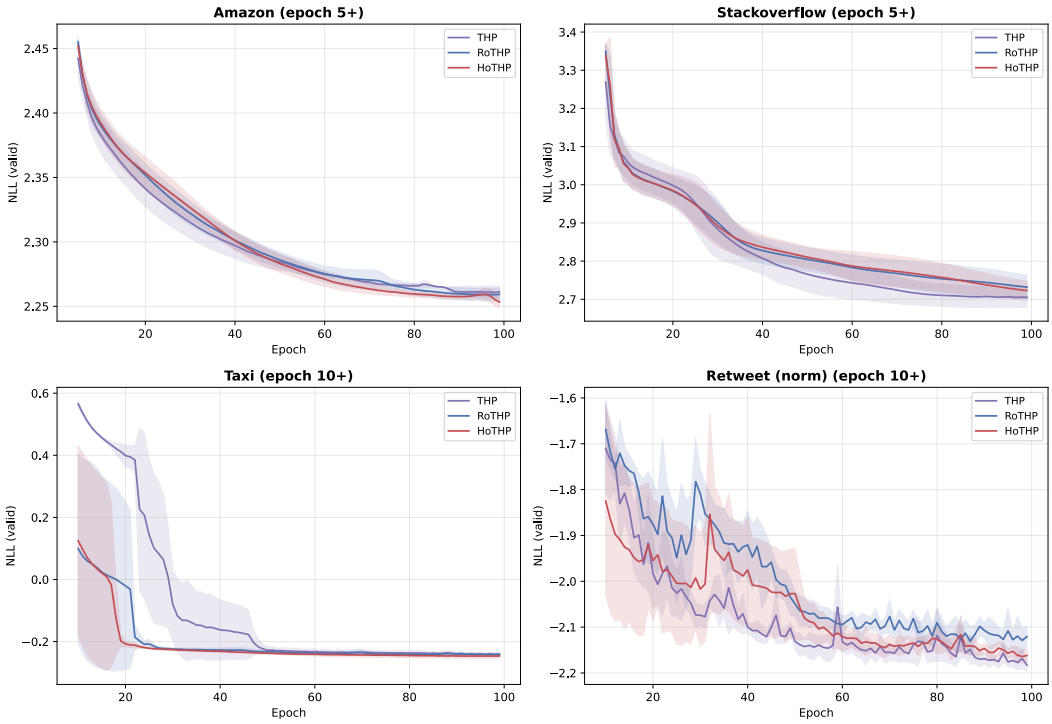
stays higher for a while and only catches up after about epoch 45. In the first epochs (10 to 20) the seeds disagree a lot for every model, which is normal on a dataset this short. For Retweet we use the normalized version (Section 5.4), because with the raw timestamps the training breaks down numerically. Once normalized, the three models converge nicely to NLL ≈ -2.1: the seeds spread out more in the early epochs (10 to 50) and the bands close up after about epoch 55. This smooth convergence, with no collapse, is the visual side of the ablation in Table 11, which shows that normalizing each sequence removes the numerical instability. Apart from Taxi, where the curves are clearly flat by the end, some of these validation curves still show a small downward slope at epoch 100, as already noted for Amazon. A longer training budget could therefore lower the absolute NLL a little, but since all the models share the same fixed budget of 100 epochs and their ordering is already stable well before that point, this does not affect the comparison or the conclusions.

5.3 Intensities learned by the models

To understand why the HoTHP behaves differently on each dataset, we extracted the conditional intensity function $\lambda^*(t)$ that each trained model computes for representative test sequences. Figure 16 shows the total intensity $\lambda^*(t) = \sum_k \lambda_k^*(t)$ over time for the Amazon, Stackoverflow, and Retweet datasets.

On Amazon, the intensity grows between events and falls when an event occurs,

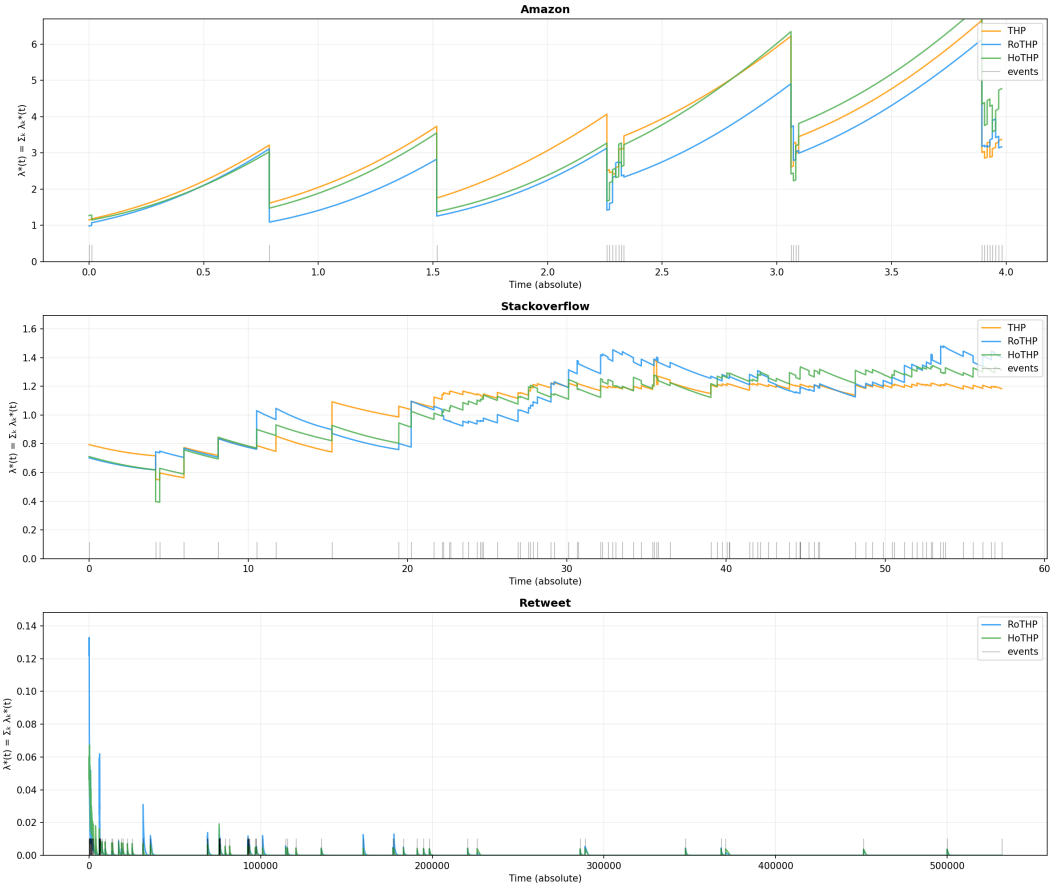
Figure 15 – Convergence curves of the validation NLL (mean ± standard deviation, 3 seeds). The first epochs were omitted to improve the visualization. On Amazon and Stackoverflow, all the models converge smoothly. On Taxi, the THP converges more slowly and only reaches the level of the other models after about epoch 45. On Retweet we use the normalized variant (Section 5.4): the three models converge smoothly to informative values, in contrast with the numerical collapse observed on the raw timestamps.



indicating that each event reduces the chance of new events in the short term. Even so, the most recent event is the most informative for the prediction, which explains why the recency bias of the HoTHP helps on this dataset. On Stackoverflow, the three models learn a practically flat intensity, with no visible reaction to the events, characteristic of a process with little temporal memory, and no positional bias manages to stand out. On Retweet, the models only react to the initial burst and then the intensity falls to practically zero, because the intervals between events are too large for any decay function.

The observation on Amazon is particularly relevant in light of Section 4.8: the HoTHP wins on this dataset even though the process is self-inhibiting, not self-exciting. This confirms that the recency bias acts on the attention ("which past events are more relevant") and not on the intensity ("the intensity rises or falls after an event"). On Amazon, the most recent event is the most informative for predicting the next one and the exponential decay in the attention captures this correctly.

Figure 16 – Conditional intensity $\lambda^*(t)$ learned by the THP (orange), RoTHP (blue), and HoTHP (green) for one test sequence of each dataset. The gray vertical bars indicate the events. On Amazon, the intensity grows between events and falls when an event occurs (self-inhibition). On Stackoverflow, the intensity is approximately constant (memoryless process). On Retweet, both models collapse to $\lambda^* \approx 0$ after the initial burst due to the extreme temporal scale.



5.4 Ablation: temporal normalization of Retweet

The numerical collapse observed on Retweet raises a question: is the problem the temporal scale or the HoTHP bias itself? To isolate these factors, we ran an ablation in which we normalize the Retweet timestamps before training.

The normalization consists of dividing all the instants of each sequence by the mean of the intervals of that sequence.

After this operation, the mean interval between events becomes approximately 1 in all the sequences, with the maximum interval reduced from 582,928 to approximately 227. The temporal structure (order of the events, clustering patterns, types) is preserved. Only the numerical scale changes.

We trained the THP, the RoTHP, and the HoTHP on the normalized Retweet with

the same hyperparameters (3 seeds, 100 epochs, $L_{\text{train}} = 50$, $L_{5\times} = 250$) and compared with the original results.

Table 11 – Ablation: effect of the temporal normalization on Retweet. "Original (raw)" uses the raw timestamps; "Normalized" uses the timestamps divided by the mean of the gaps. Mean ± standard deviation (3 seeds). The best value in each column of the normalized block is in bold and the second best is underlined.

Version	Model	NLL 1×	NLL 2×	NLL 5×	Acc 5×
Original (raw)	THP	15.94	3.6×10^3	2.7×10^3	0.491
	RoTHP	15.94	6.7×10^5	7.0×10^5	0.491
	HoTHP	15.94	1.2×10^5	1.4×10^5	0.491
Normalized	THP	$\mathbf{-2.180 \pm 0.008}$	$\underline{-1.227 \pm 0.053}$	$\underline{0.039 \pm 0.332}$	$\underline{0.597 \pm 0.002}$
	RoTHP	-2.133 ± 0.021	0.749 ± 0.618	47.563 ± 8.703	0.589 ± 0.002
	HoTHP	$\underline{-2.139 \pm 0.014}$	$\mathbf{-1.275 \pm 0.013}$	$\mathbf{-0.651 \pm 0.013}$	$\mathbf{0.600 \pm 0.002}$

The normalization clearly fixes the numerical collapse. With raw timestamps, the NLL collapses to ≈ 15.9 already at $1\times$ (the same value for the three models, a clear sign of saturation) and grows to the order of 10^4–10^5 at $5\times$.

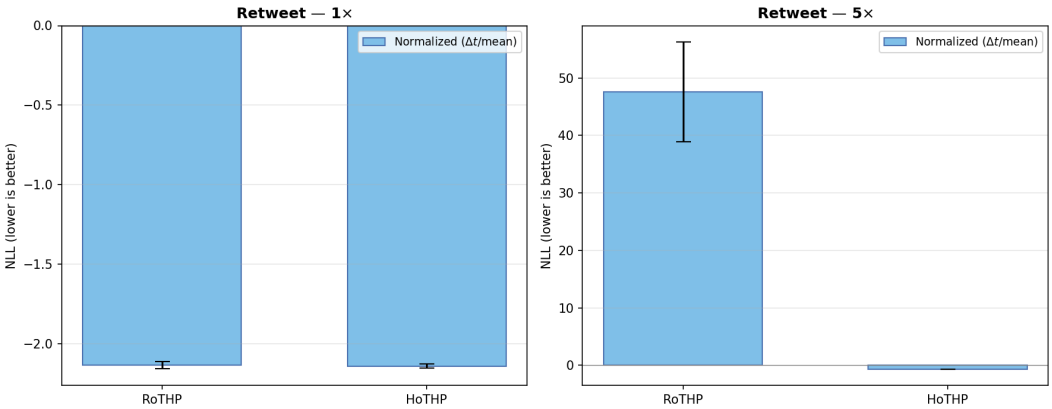
The three models collapse to the same value, which is informative: the problem appears before the kernel matters. On the raw data the intervals reach up to 582,928 time units. Each model turns time into numbers with functions that only work in a limited range (the sinusoidal encoding of the THP, the rotary rotation of the RoTHP, and the hyperbolic kernel of the HoTHP). Numbers this large push all of them out of that range, so the functions either oscillate too fast and become noise, or they saturate. The models then lose track of time and fall back to the same guess, a nearly constant rate. This already happens at $1\times$, at the training length itself, so it is a numerical effect of the timestamp scale, shared by the three models regardless of their kernel.

With normalized timestamps, the NLL at $1\times$ drops to negative values and the training converges across all the seeds.

The difference between the models appears in the extrapolation, as illustrated in Figure 17. The normalized HoTHP obtains NLL $5\times = -0.651 \pm 0.013$, practically invariant between seeds, and it is the only model that keeps a negative NLL in extrapolation. The THP degrades to ≈ 0.04 (with high variance, ± 0.33) and the RoTHP degrades much more, to 47.6 ± 8.7. The type accuracy follows the same pattern: the normalized HoTHP keeps the best Acc $5\times = 0.600 \pm 0.002$, against 0.597 of the THP and 0.589 of the RoTHP.

These results make it possible to separate the two factors that hurt the performance

Figure 17 – NLL on the normalized Retweet (mean ± standard deviation, 3 seeds). The figure compares the RoTHP and the HoTHP, the two rotary-kernel models. On the left, the 1× condition (training length): the RoTHP and the HoTHP obtain a similar NLL. On the right, the 5× condition (extrapolation): the HoTHP keeps a stable NLL, while the RoTHP shows high variance due to one seed that diverges.



on Retweet. The primary problem was the numerical scale. The normalization benefits all the models drastically. But even with the scale corrected, the HoTHP surpasses both the THP and the RoTHP in extrapolation, confirming that the recency bias is advantageous on this dataset when the temporal scale allows its calibration. The residual degradation of the RoTHP (NLL $5\times = 47.6$, far from the HoTHP) reinforces the fragility of the oscillatory kernel in extrapolation, also observed in the synthetic data.

5.5 Discussion

The results on real data show that the recency bias of the HoTHP is beneficial in some scenarios and neutral or harmful in others. The analysis of the learned intensities suggests that the determining factor is not whether the process is self-exciting or self-inhibiting, but whether recent events are more informative than distant ones for the construction of the hidden state.

On Amazon, where the HoTHP obtains the best result, the process is self-inhibiting: the intensity grows with the time since the last event. Even so, the recency bias helps, because the immediately previous event is the most relevant for predicting the next one. The exponential decay in the attention correctly captures this relationship of decreasing relevance.

On Stackoverflow, where the three models tie, the intensity is practically constant. A rate that stays flat and does not react to the history points to a process closer to a homogeneous Poisson process, or to a Cox process if the rate varies slowly for reasons external to the events,

than to a self-exciting Hawkes process. This indicates a process with little or no temporal memory, in which all the past events are equally (little) informative. The recency bias of the HoTHP does not significantly hurt (the constant intensity shows that the model manages to compensate in the intensity layer), but it also does not help, because there is no temporal structure to exploit.

On Retweet, the problem is twofold: the extreme temporal scale causes numerical collapse, and the structure of viral cascades may not align with the exponential decay. The ablation in Section 5.4 separated these two factors and showed that the primary problem was the scale: with normalization, the HoTHP obtains the best result of the whole benchmark (NLL $5\times = -0.651$), with almost zero variance between seeds. The RoTHP, even with normalization, still degrades sharply in the extrapolation (NLL $5\times = 47.6$).

The HoTHP proves more effective when two conditions are satisfied at the same time: (i) the process has short-range temporal memory, so that recent events are more informative than distant ones; and (ii) the scale of the intervals between events allows the parameter θ' to be calibrated properly during training, without numerical saturation. The Retweet ablation confirms that condition (ii) can be solved by preprocessing, and when it is, the recency bias of the HoTHP proves consistently advantageous.

5.5.1 Recency bias and long-range memory

A slower memory is not, in itself, a problem for the HoTHP. Both synthetic scenarios are decaying processes, and the HoTHP wins both at every factor beyond the training length, because the decay rate θ' is learnable and can be pushed low enough to keep attending to older events. What the slow decay does show is that extrapolation gets harder for every model as the memory lengthens: the NLL of the HoTHP grows by 0.819 between $1\times$ and $10\times$, against only 0.085 in the fast decay, with higher variance between seeds (± 0.625). The HoTHP still keeps the best NLL, with a lead over the baselines slightly wider than in the fast decay. But its curve is less flat, a sign that a single global θ' calibrated on 50-event sequences is harder to set well when the relevant range extends far beyond training.

The real limit of the recency bias is a different regime. When the dependence is not monotonic, for example when distant events are as informative as recent ones, or when the pattern is periodic, no value of θ' can help, because the kernel can only decrease with the distance. The *train-short, test-long* protocol also rewards a recency bias almost by construction:

local structure is invariant to the sequence length, while a dependency spanning hundreds of events is never seen in a 50-event training and cannot be learned or tested under it. The HoTHP is therefore the right tool for processes whose relevance decays with time, and not for those where the relevant structure is flat or periodic. A natural direction for future work is to combine the recency bias with a mechanism able to retain a few selected distant events, keeping the extrapolation stability without giving up long-range memory.

5.5.2 Periodic processes

A specific case where a purely decaying kernel is a poor fit is a periodic process. Many real event sequences have a natural cycle: taxi rides follow the daily and weekly rhythm of the city, server logs follow the working day, human activity follows sleep and routine. In a process like this, the most informative past event for predicting the next one is often not the most recent event, but the event that happened roughly one period ago, such as the same hour yesterday or the same day last week.

This is exactly the situation the recency bias of the HoTHP cannot represent. The hyperbolic kernel is built from decaying exponentials, so the weight it assigns to a past event only falls as the temporal distance grows. It has no way to raise the weight again at a specific distance, which is what a periodic pattern would require: a peak of attention at one period, another at two periods, and so on. No value of θ' produces such a peak, because θ' only controls how fast the single decay falls, not where it rises. A model with a monotonic recency bias will always prefer the most recent event, even when the useful signal is one full cycle behind.

A natural fix, and a direction for future work, is to add a periodic term to the kernel, so that the attention weight is the sum of the hyperbolic decay and a periodic component. The period would be a learnable parameter, aligned with the real cycle of the process. Note that the trigonometric kernel of the RoTHP is already periodic, but it does not solve this on its own, because its period is not tied to the real cycle and, in extrapolation, it falls on phases not seen in training.

5.5.3 Computational cost versus benefit in extrapolation

The hyperbolic kernel of the HoTHP involves more operations than the trigonometric kernel of the RoTHP, which is reflected in the training time. Table 12 presents the average training times (100 epochs, 3 seeds) for each combination of dataset and model.

Table 12 – Average training time in minutes (mean over 3 seeds, except NHP on Amazon with 2 seeds; NHP was not run on the normalized Retweet). All the models use the same architecture size and training budget (100 epochs).

Dataset	THP	RoTHP	HoTHP	NHP
Taxi	1.9	1.9	2.0	2.7
Stackoverflow	2.5	2.5	3.2	4.9
Amazon	9.4	9.5	12.2	17.7
Retweet (norm)	34.1	34.4	83.2	—

Compared with the THP and the RoTHP, the HoTHP is between 1.07× (Taxi) and 2.44× (normalized Retweet) slower, the overhead being more pronounced on datasets with long sequences, where the computation of the hyperbolic kernel dominates the time per batch. For datasets of moderate length (Taxi, Stackoverflow, Amazon), the additional cost stays around 30% or below. The NHP, the strongest baseline in NLL on these datasets, is also the most expensive model of all: it is 1.3× to 1.5× slower than the HoTHP (for example, 17.7 against 12.2 minutes on Amazon). The recency-bias kernel therefore adds a moderate cost over the other Transformers while remaining cheaper than the recurrent alternative.

This cost, however, is paid only once during training. At inference time, the trained model is applied to sequences that are potentially much longer than the ones seen in training, exactly the extrapolation scenario. In this context, what matters is not the training speed, but whether the model produces valid results. Table 13 reports this inference time. The results in Table 11 illustrate the point clearly: on the normalized Retweet with 5× extrapolation, the RoTHP obtains NLL $= 47.6 \pm 8.7$, while the HoTHP obtains NLL $= -0.651 \pm 0.013$. Training the RoTHP 2.4× faster only to then obtain unusable results in extrapolation does not constitute a practical advantage.

Table 13 – Inference time in milliseconds (mean ± standard deviation, 3 seeds). Time of a single forward pass over the test set. Lower is better.

Dataset	Model	1×	2×	5×
Taxi	THP	246.9 ± 61.0	205.2 ± 7.9	239.5 ± 53.8
	RoTHP	206.2 ± 11.1	258.0 ± 71.9	244.3 ± 66.6
	HoTHP	256.7 ± 55.5	252.1 ± 56.0	214.4 ± 6.8
	NHP	272.1 ± 56.2	296.4 ± 66.9	230.5 ± 15.5
Stackoverflow	THP	276.8 ± 62.0	306.2 ± 80.1	231.8 ± 7.5
	RoTHP	257.9 ± 44.1	244.8 ± 13.5	281.8 ± 61.6
	HoTHP	500.6 ± 16.7	377.3 ± 10.0	374.2 ± 9.1
	NHP	440.5 ± 85.8	318.9 ± 8.7	321.4 ± 11.2
Amazon	THP	1260.6 ± 255.5	1177.6 ± 230.4	1018.4 ± 17.8
	RoTHP	1033.8 ± 29.3	1156.6 ± 224.0	1281.9 ± 242.9
	HoTHP	1675.7 ± 217.4	1637.7 ± 206.4	1636.9 ± 192.1
	NHP	1392.7 ± 236.4	1418.8 ± 236.9	1489.3 ± 283.2
Retweet (norm)	THP	1806.7 ± 280.0	1804.2 ± 290.1	1821.9 ± 311.2
	RoTHP	1825.5 ± 272.1	1834.9 ± 294.9	1817.9 ± 276.6
	HoTHP	6504.9 ± 197.1	6479.6 ± 211.9	6484.9 ± 125.1

6 CONCLUSIONS AND FUTURE WORK

This work proposed the HoTHP, a variant of the Transformer Hawkes Process that replaces the trigonometric kernel of the RoTHP with a hyperbolic kernel that has exponential decay. The motivation was to improve length extrapolation in temporal point processes: to train on short sequences and keep performance on longer sequences.

6.1 Contributions

The main contribution is the introduction of the exponential recency bias into the temporal attention mechanism. Unlike the oscillatory kernel of the RoTHP, whose attention score can grow for temporal distances not seen in training, the hyperbolic kernel of the HoTHP guarantees a monotonic decay. For any temporal distance, including values larger than those observed during training, the attention score is bounded and tends to zero. This allows the model to generalize to longer sequences without degradation.

Besides the architectural proposal, the experimental analysis revealed two observations that we consider relevant for the field:

1. The exponential bias acts exclusively on the attention mechanism, which builds the hidden state from the past events. The intensity function is a separate layer with free learnable parameters. This means that the HoTHP can model both self-exciting and self-inhibiting processes. What the bias imposes is not the shape of the intensity, but the shape of the relevance: recent events receive more weight in the construction of the internal representation.
2. The visualization of the learned intensities showed that the Amazon dataset exhibits a self-inhibiting pattern (the intensity grows between events and falls when an event occurs), while Stackoverflow exhibits an almost constant intensity (no temporal memory). The HoTHP wins on the first and ties on the second, confirming that the relevant factor is the presence of short-range temporal memory, regardless of the direction (excitation or inhibition).

6.2 Main results

On the synthetic data generated by Hawkes processes with an exponential kernel, the HoTHP obtains the best NLL at all extrapolation factors ($2\times$, $5\times$, and $10\times$) in both decay

regimes, surpassing all the baselines (NHP, THP, and RoTHP); the advantage is statistically significant (paired, one-sided Wilcoxon test) in all cases, with the slow decay at $10\times$ against the NHP being the narrowest-margin comparison. This result was expected, since the bias of the model is perfectly aligned with the generating process.

On the real data, the results vary by dataset. The HoTHP obtains the best performance on Amazon ($\Delta\text{NLL} = -0.074$ in the $5\times$ extrapolation, against -0.020 of the RoTHP), ties on Stackoverflow and Taxi, and fails on Retweet with raw data due to the extreme temporal scale of the dataset. The ablation with temporal normalization revealed that the Retweet problem was primarily one of numerical scale: with normalized timestamps, the HoTHP obtains an NLL $5\times$ of -0.651 ± 0.013, the best result among all the models and conditions tested, with almost zero variance between seeds. The analysis of these results points to two conditions necessary for the bias to be beneficial: the process needs to have short-range temporal memory, and the scale of the intervals between events needs to be in a range that allows the calibration of the parameter θ'.

From the computational point of view, the HoTHP is between $1.05\times$ and $2.24\times$ slower in training, depending on the length of the sequences. This additional cost is paid only once and is justified by the benefit in extrapolation: the trained model produces stable results on sequences up to $5\times$ longer, while the RoTHP can collapse numerically in this same scenario.

6.3 Discussion

The experiments also give a more balanced picture of what the Transformer brings to point process modeling. On the positive side, attention removes the sequential bottleneck of the recurrent models: all the events are processed in parallel, which makes training faster on long sequences. Our own timing measurements show this clearly, since the recurrent NHP is the slowest model of all, between $1.3\times$ and $1.5\times$ slower than the HoTHP. Attention also relates any two events directly, without carrying information through a single hidden state, and it offers a natural place to inject a temporal inductive bias, which is exactly what the HoTHP does through its kernel. On the negative side, attention has no native notion of continuous time. By itself it is invariant to the order of the events, so the temporal structure has to be added through the positional encoding. The choice of this encoding is decisive, as the gap between the THP, the RoTHP, and the HoTHP shows. The attention cost is also quadratic in the length of the sequence, against the linear cost of a recurrent model. And, on real data, the recurrent NHP still obtains the best likelihood on several datasets. This is not a difference in the probabilistic model. All

of these models describe the same object, the conditional intensity of a temporal point process trained by maximum likelihood, and differ only in the network architecture that computes it. The advantage of the NHP is architectural: its continuous-time recurrence represents the intensity between events through a hidden state that decays continuously, while the attention models build it from the representations at the event times only.

Taken together, the results suggest that there is no single best model, and that the choice depends on the goal. When the priority is to train on a large volume of data or on long sequences, the parallelism of the Transformer is the decisive advantage. When the goal is to predict the type of the next event, the HoTHP obtains the best accuracy, surpassing even the recurrent baseline on Amazon and Stackoverflow. When the priority is stable performance on sequences much longer than those seen in training, the recency bias of the HoTHP is what keeps the model from degrading. And when the goal is only the likelihood on a real dataset with an unknown structure, the recurrent NHP tends to win, unless the process has the kind of decaying memory that the HoTHP is built for, as in the synthetic experiments and under extrapolation, where the HoTHP obtains the best NLL as well. The contribution of this work should be read in this light: not as a model that dominates every metric, but as one that makes the Transformer respect the temporal decay structure of the Hawkes process and, with it, extrapolate in a stable way.

6.4 Limitations

The work has some limitations that should be considered:

1. The evaluation on real data is limited to four datasets. A broader evaluation, including datasets from domains such as seismology, finance, and computer networks, would strengthen the conclusions about when the bias is beneficial.
2. The HoTHP does not have an internal mechanism for temporal normalization. The scale of the timestamps is handled only by the shift to zero ($\tilde{t}_i = t_i - t_1$), which preserves the differences but does not control the absolute magnitude. Datasets with very large intervals (such as Retweet) cause numerical saturation in the kernel.
3. The parameter θ' controls a single global decay rate. Processes with multiple temporal scales may require multiple decay rates.

6.5 Future work

The limitations listed above point to natural extensions of the HoTHP. In addition, the recent literature on point processes and on positional encodings offers alternatives that were not explored in this work and that we consider promising directions. We organize the future work in five points:

1. **Multiple decay scales.** The parameter θ' imposes a single global decay rate. A direct extension is to learn one decay rate per attention head, in the spirit of the per-head slopes of ALiBi (PRESS *et al.*, 2021), so that each head can specialize in a different temporal scale.
2. **Alternative decay shapes.** The hyperbolic kernel imposes an exponential decay, but many real processes exhibit power-law decay, as in seismic aftershocks and social cascades. Attention biases with a power-law shape have already shown good results in other domains (SHEN *et al.*, 2025).
3. **Broader evaluation.** The evaluation could be extended to domains such as seismology, finance, and computer networks, and to standardized long-horizon benchmarks such as HoTPP (KARPUKHIN *et al.*, 2024), which measures exactly the regime that motivates this work: prediction far beyond the observed history.
4. **State space models.** A recent line of work replaces attention entirely with selective state space models. The Mamba Hawkes Process (GAO *et al.*, 2024), proposed by the same authors of the RoTHP, encodes the event history with a cost that is linear in the length of the sequence, against the quadratic cost of attention. Since the recency bias of the HoTHP is a property of the kernel and not of the Transformer itself, an open question is whether this bias transfers to these architectures, and how the HoTHP compares against them under extrapolation.
5. **Large language models.** Another recent direction uses pretrained language models as the backbone for event sequence modeling. In this approach, the event types are represented as text, so the model can use the semantic content of the events instead of treating them as opaque categories, while the timestamps are injected through temporal embeddings (LIU; QUAN, 2025). These models inherit the positional encoding of the language model, which was designed for discrete text positions and not for continuous time. Studying how a temporal bias such as the hyperbolic kernel could be adapted to this setting is a natural continuation of this work.

BIBLIOGRAPHY

DAI, C.; SHAN, H.; SONG, M.; LIANG, D. **HoPE: Hyperbolic Rotary Positional Encoding for Stable Long-Range Dependency Modeling in Large Language Models**. arXiv, 2025. ArXiv:2509.05218 [cs]. Disponível em: <http://arxiv.org/abs/2509.05218>.

DAI, S.; GAO, A.; LI, Z.; DU, Y. **Rotary Position Embedding-Based Transformer Hawkes Process for Event-Type Big Data**. 2026. Disponível em: <https://www.sciopen.com/article/10.26599/BDMA.2025.9020029>.

DALEY, D. J.; VERE-JONES, D. **An Introduction to the Theory of Point Processes: Volume I: Elementary Theory and Methods**. [S.l.]: Springer Science & Business Media, 2003. Google-Books-ID: Ev2iXQKItpUC. ISBN 978-0-387-95541-4.

DU, N.; DAI, H.; TRIVEDI, R.; UPADHYAY, U.; GOMEZ-RODRIGUEZ, M.; SONG, L. Recurrent Marked Temporal Point Processes: Embedding Event History to Vector. In: **Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining**. New York, NY, USA: Association for Computing Machinery, 2016. (KDD '16), p. 1555–1564. ISBN 978-1-4503-4232-2. Disponível em: <https://doi.org/10.1145/2939672.2939875>.

GAO, A.; DAI, S.; HU, Y. **Mamba Hawkes Process**. arXiv, 2024. ArXiv:2407.05302 [cs.LG]. Disponível em: <http://arxiv.org/abs/2407.05302>.

HAWKES, A. G. Spectra of some self-exciting and mutually exciting point processes. **Biometrika**, v. 58, n. 1, p. 83–90, abr. 1971. ISSN 0006-3444. Disponível em: <https://doi.org/10.1093/biomet/58.1.83>.

KARPUKHIN, I.; SHIPILOV, F.; SAVCHENKO, A. HoTPP Benchmark: Are We Good at the Long Horizon Events Forecasting? out. 2024. Disponível em: <https://openreview.net/forum?id=1yJ3IDpb1D>.

LIU, Z.; QUAN, Y. **TPP-LLM: Modeling Temporal Point Processes by Efficiently Fine-Tuning Large Language Models**. arXiv, 2025. ArXiv:2410.02062 [cs.LG]. Disponível em: <http://arxiv.org/abs/2410.02062>.

MEI, H.; EISNER, J. **The Neural Hawkes Process: A Neurally Self-Modulating Multivariate Point Process**. arXiv, 2017. ArXiv:1612.09328 [cs]. Disponível em: <http://arxiv.org/abs/1612.09328>.

PRESS, O.; SMITH, N.; LEWIS, M. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. In: . [s.n.], 2021. Disponível em: <https://openreview.net/forum?id=R8sQPpGCv0>.

RASMUSSEN, J. G. **Lecture Notes: Temporal Point Processes and the Conditional Intensity Function**. arXiv, 2018. ArXiv:1806.00221 [stat]. Disponível em: <http://arxiv.org/abs/1806.00221>.

ROSS, S. M. **A first course in probability**. Tenth edition. Boston: Pearson, 2019. ISBN 978-0-13-475311-9.

SHCHUR, O.; TüRKMEN, A. C.; JANUSCHOWSKI, T.; GüNNEMANN, S. **Neural Temporal Point Processes: A Review**. arXiv, 2021. ArXiv:2104.03528 [cs]. Disponível em: <http://arxiv.org/abs/2104.03528>.

SHEN, J.; TANG, Y.; FERGUSON, A. **Power law attention biases for molecular transformers**. arXiv, 2025. ArXiv:2511.11489 [physics] version: 1. Disponível em: <http://arxiv.org/abs/2511.11489>.

SU, J.; LU, Y.; PAN, S.; MURTADHA, A.; WEN, B.; LIU, Y. **RoFormer: Enhanced Transformer with Rotary Position Embedding**. arXiv, 2023. ArXiv:2104.09864 [cs]. Disponível em: <http://arxiv.org/abs/2104.09864>.

VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, u.; POLOSUKHIN, I. Attention is All you Need. In: GUYON, I.; LUXBURG, U. V.; BENGIO, S.; WALLACH, H.; FERGUS, R.; VISHWANATHAN, S.; GARNETT, R. (Ed.). **Advances in Neural Information Processing Systems**. Curran Associates, Inc., 2017. v. 30. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>.

XUE, S.; SHI, X.; CHU, Z.; WANG, Y.; HAO, H.; ZHOU, F.; JIANG, C.; PAN, C.; ZHANG, J. Y.; WEN, Q.; ZHOU, J.; MEI, H. **EasyTPP: Towards Open Benchmarking Temporal Point Processes**. arXiv, 2024. ArXiv:2307.08097 [cs]. Disponível em: <http://arxiv.org/abs/2307.08097>.

ZHANG, Q.; LIPANI, A.; KIRNAP, O.; YILMAZ, E. Self-Attentive Hawkes Process. In: **Proceedings of the 37th International Conference on Machine Learning**. PMLR, 2020. p. 11183–11193. ISSN 2640-3498. Disponível em: <https://proceedings.mlr.press/v119/zhang20q.html>.

ZHOU, F.; KONG, Q.; QIAO, J.; WAN, C.; ZHANG, Y.; CAI, R. **Advances in Temporal Point Processes: Bayesian, Neural, and LLM Approaches**. arXiv, 2025. ArXiv:2501.14291 [cs]. Disponível em: <http://arxiv.org/abs/2501.14291>.

ZUO, S.; JIANG, H.; LI, Z.; ZHAO, T.; ZHA, H. **Transformer Hawkes Process**. arXiv, 2021. ArXiv:2002.09291 [cs]. Disponível em: <http://arxiv.org/abs/2002.09291>.

16% geral de similaridade

As principais fontes encontradas nos seguintes bancos de dados:

- 12% Banco de dados da Internet
- Banco de dados do Crossref
- 10% Banco de dados de trabalhos enviados
- 10% Banco de dados de publicações
- Banco de dados de conteúdo publicado no Crossref

PRINCIPAIS FONTES

As fontes com o maior número de correspondências no envio. Fontes sobrepostas não serão exibidas.

1	**arxiv.org** Internet	3%
2	**export.arxiv.org** Internet	<1%
3	**jhir.library.jhu.edu** Internet	<1%
4	**repositorio.ufc.br** Internet	<1%
5	**University Politehnica of Bucharest on 2026-06-20** Submitted works	<1%
6	**dx.doi.org** Internet	<1%
7	**raw.githubusercontent.com** Internet	<1%
8	**Marcel Rodrigues de Barros. "Incorporando as irregularidades de dado...** Crossref posted content	<1%

9	Alana da Silva Luna. "AllianceScan: uma abordagem para identificação... Crossref posted content	<1%
10	mdpi.com Internet	<1%
11	jku on 2025-07-23 Submitted works	<1%
12	jscholarship.library.jhu.edu Internet	<1%
13	Universidade Federal do Ceará, UFC on 2022-05-24 Submitted works	<1%
14	proceedings.mlr.press Internet	<1%
15	mdpi-res.com Internet	<1%
16	proceedings.iclr.cc Internet	<1%
17	Press, Ofir. "Complementing Scale: Novel Guidance Methods for Impro... Publication	<1%
18	University College Roosevelt on 2025-07-28 Submitted works	<1%
19	escholarship.org Internet	<1%
20	Aston University on 2023-09-28 Submitted works	<1%

21	"Advanced Intelligent Computing Technology and Applications", Spring... Crossref	<1%
22	Ali Khodadadi, Seyed Abbas Hosseini, Erfan Tavakoli, Hamid R. Rabiee.... Crossref	<1%
23	Chang, Yuxin. "Flexible and Efficient Machine Learning Models for Mar... Publication	<1%
24	Universidade Federal do Ceará, UFC on 2022-11-01 Submitted works	<1%
25	hal.science Internet	<1%
26	themoonlight.io Internet	<1%
27	University of Oxford on 2026-07-02 Submitted works	<1%
28	Indian Institute of Technology on 2022-07-24 Submitted works	<1%
29	sh-tsang.medium.com Internet	<1%
30	proceedings.com Internet	<1%
31	Indian Institute of Technology on 2022-06-29 Submitted works	<1%
32	sribd.cn Internet	<1%

33	Knight, Nicole Katherine Sutts. "Linking the Feeding Ecology of Marine ... Publication	<1%
34	assets.amazon.science Internet	<1%
35	"ECAI 2020", IOS Press, 2020 Crossref	<1%
36	P.N. Paraskevopoulos. "Modern Control Engineering", CRC Press, 2017 Publication	<1%
37	arxiv-vanity.com Internet	<1%
38	coursehero.com Internet	<1%
39	Yin Li. "Theoretical Analysis of Positional Encodings in Transformer M... Crossref posted content	<1%
40	Hongyun Cai, Thanh Tung Nguyen, Yan Li, Vincent W. Zheng, Binbin Ch... Crossref	<1%
41	Indian Institute of Technology, Bombay on 2019-04-08 Submitted works	<1%
42	Liverpool John Moores University on 2023-12-04 Submitted works	<1%
43	research-collection.ethz.ch Internet	<1%
44	Indian Institute of Science, Bangalore on 2025-06-25 Submitted works	<1%

45	egusphere.copernicus.org Internet	<1%
46	scielo.br Internet	<1%
47	teses.usp.br Internet	<1%
48	Angelica Liguori, Luciano Caroprese, Marco Minici, Bruno Veloso, Fran... Crossref	<1%
49	Imperial College of Science, Technology and Medicine on 2021-09-03 Submitted works	<1%
50	Massey University on 2017-03-14 Submitted works	<1%
51	University of Warwick on 2016-05-17 Submitted works	<1%
52	Y.K. Cheung, J.H.W. Lee, A.Y.T. Leung. "Computational Mechanics, Vol... Publication	<1%
53	hdl.handle.net Internet	<1%
54	pleasedameunkiss.blogspot.com Internet	<1%
55	web.realinfo.tv Internet	<1%
56	frontiersin.org Internet	<1%

57	Indian Institute of Science, Bangalore on 2024-06-21 Submitted works	<1%
58	Qiang Zhang, Aldo Lipani, Emine Yilmaz. "Learning Neural Point Proces... Crossref	<1%
59	Universidade de Sao Paulo on 2018-09-28 Submitted works	<1%
60	ynu.repo.nii.ac.jp Internet	<1%
61	Soheil Amanipour, Mohammad Kazem Sheikh-El-Eslami, Hamid Reza B... Crossref	<1%
62	University of Hertfordshire on 2023-01-08 Submitted works	<1%
63	Ain Shams University on 2020-08-23 Submitted works	<1%
64	University College London on 2018-09-03 Submitted works	<1%
65	University of the Western Cape on 2026-05-20 Submitted works	<1%
66	10xsheets.com Internet	<1%
67	politesi.polimi.it Internet	<1%
68	Birla Institute of Technology and Science Pilani on 2021-06-21 Submitted works	<1%

69	Flavio Fagundes Ferreira. "Seleção de modelos de associação RC utiliz... Crossref posted content	<1%
70	papers.nips.cc Internet	<1%
71	repositorio.ufscar.br Internet	<1%
72	cc.ufrrj.br Internet	<1%
73	Concordia University on 2026-08-14 Submitted works	<1%
74	Lei Jiang, Huan Liu, Jesse S. Jin, Yu Zheng, Xin He, Jie Zhao. "TransSe... Crossref	<1%
75	Pereira, Kleiton. "Scheduling HPC Jobs with Graph Neural Networks", U... Publication	<1%
76	University of Illinois at Urbana-Champaign on 2022-02-02 Submitted works	<1%
77	artemis.cslab.ece.ntua.gr:8080 Internet	<1%
78	"Combinatorial Optimization and Applications", Springer Science and B... Crossref	<1%
79	Daniel A. Griffith. "Beyond Mule Kicks: The Poisson Distribution in Geo... Crossref	<1%
80	Heriot-Watt University on 2023-11-29 Submitted works	<1%

81	Sam Boshar, Evan Trop, Bernardo P de Almeida, Liviu Copoiu, Thomas ... Crossref	<1%
82	University of Leeds on 2025-08-26 Submitted works	<1%
83	comum.rcaap.pt Internet	<1%
84	biorxiv.org Internet	<1%
85	Indiana University on 2023-03-21 Submitted works	<1%
86	Lars Schäfer. "<mml:math altimg="si1.gif" overflow="scroll" xmlns... Crossref	<1%
87	Symbiosis International University on 2017-09-20 Submitted works	<1%
88	Tatsunori Tanaka, Fi Zheng, Kai Sato, Zhifeng Li, Shi Li. "Time-Aware P... Crossref posted content	<1%
89	Universidade do Porto on 2019-07-22 Submitted works	<1%
90	University College London on 2024-01-20 Submitted works	<1%
91	University of Birmingham on 2021-09-13 Submitted works	<1%
92	docslib.org Internet	<1%

93	mason.gmu.edu Internet	<1%
94	statswithr.com Internet	<1%
95	"Advanced Analytics and Learning on Temporal Data", Springer Scienc... Crossref	<1%
96	"Database Systems for Advanced Applications", Springer Science and ... Crossref	<1%
97	Feng , Liang. "Importance Sampling of Affine Jump Diffusions and Rela... Publication	<1%
98	Hojjat Karami, David Atienza, Anisoara Ionescu. "TEE4EHR: Transform... Crossref	<1%
99	Imperial College of Science, Technology and Medicine on 2022-09-02 Submitted works	<1%
100	Irina Abdullaeva, Ivan Karpukhin, Andrei Filatov, Mikhail Orlov et al. "ES... Crossref	<1%
101	Marian-Andrei Rizoiu, Swapnil Mishra, Quyu Kong, Mark Carman, Lexin... Crossref	<1%
102	Shen, Tianyi. "Towards Scalable and Efficient Deep Learning Models.", ... Publication	<1%
103	The University of Manchester on 2024-04-19 Submitted works	<1%
104	Universitat Politècnica de València on 2018-07-07 Submitted works	<1%

105	Universiteit van Amsterdam on 2018-09-28 Submitted works	<1%
106	University of Edinburgh on 2019-01-14 Submitted works	<1%
107	University of Edinburgh on 2024-04-25 Submitted works	<1%
108	University of Hong Kong on 2025-11-11 Submitted works	<1%
109	University of Sheffield on 2025-06-15 Submitted works	<1%
110	University of Warwick on 2014-05-01 Submitted works	<1%
111	University of Warwick on 2017-05-11 Submitted works	<1%
112	discovery-pp.ucl.ac.uk Internet	<1%
113	expeditiorepositorio.utadeo.edu.co Internet	<1%
114	scholarbank.nus.edu.sg Internet	<1%
115	sciopen.com Internet	<1%
116	"Database Systems for Advanced Applications", Springer Science and ... Crossref	<1%

117	"Machine Learning and Knowledge Discovery in Databases. Research T... Crossref	<1%
118	Bocconi University on 2011-10-28 Submitted works	<1%
119	Heriot-Watt University on 2020-05-10 Submitted works	<1%
120	Imperial College of Science, Technology and Medicine on 2020-10-12 Submitted works	<1%
121	Imperial College of Science, Technology and Medicine on 2022-06-22 Submitted works	<1%
122	Imperial College of Science, Technology and Medicine on 2026-06-02 Submitted works	<1%
123	Indian Institute of Technology on 2019-06-20 Submitted works	<1%
124	Indiana University on 2024-02-21 Submitted works	<1%
125	Kyung Hye Kim, So Young Sohn. "Hybrid neural network with cost-sens... Crossref	<1%
126	Laila F. Seddek, Abdelhalim Ebaid, Essam R. El-Zahar, Mona D. Aljoufi. ... Crossref	<1%
127	Queen Mary and Westfield College on 2023-08-24 Submitted works	<1%
128	Shengran Guo. "Application and Impact of Position Embedding in Natu... Crossref	<1%

129	Shuting Li, Shuanghong Shen, Yu Su, Xinjie Sun, Junyu Lu, Qi Mo, Zhen... Crossref	<1%
130	University College London on 2025-04-27 Submitted works	<1%
131	University of Birmingham on 2025-08-30 Submitted works	<1%
132	University of Leeds on 2025-08-29 Submitted works	<1%
133	University of Stellenbosch, South Africa on 2024-09-06 Submitted works	<1%
134	Vrije Universiteit Amsterdam on 2026-07-22 Submitted works	<1%
135	downloads.hindawi.com Internet	<1%
136	ecmlpkdd-storage.s3.eu-central-1.amazonaws.com Internet	<1%
137	nanopdf.com Internet	<1%
138	oatao.univ-toulouse.fr Internet	<1%
139	ojs.aaai.org Internet	<1%
140	roots-automation.github.io Internet	<1%

141	numberanalytics.com Internet	<1%
142	"An Introduction to the Theory of Point Processes", Springer Nature, 20... Crossref	<1%
143	Higher Education Commission Pakistan on 2016-11-18 Submitted works	<1%
144	Liang Chen, Shyuichi Izumiya, DongHe Pei, Kentaro Saji. "Anti de Sitter ... Crossref	<1%
145	Oklahoma State University on 2015-11-07 Submitted works	<1%
146	Patrick J. Laub, Young Lee, Thomas Taimre. "The Elements of Hawkes ... Crossref	<1%
147	Pierre Del Moral, Spiridon Penev. "Stochastic Processes - From Applic... Publication	<1%
148	Sergey Edward Lyshevski. "Design and Control of Physical and Cyber-P... Publication	<1%
149	University of Birmingham on 2020-09-03 Submitted works	<1%
150	University of Melbourne on 2021-10-24 Submitted works	<1%
151	University of Newcastle upon Tyne on 2013-12-10 Submitted works	<1%
152	University of Sydney on 2024-11-02 Submitted works	<1%

153	Xu, Zhiqing. "A General-Purpose Deep Learning Framework for Protein ... Publication	<1%
154	Zhi-yan Song, Jian-wei Liu, Jie Yang, Lu-ning Zhang. "Linear normalizat... Crossref	<1%
155	"Database Systems for Advanced Applications", Springer Science and ... Crossref	<1%
156	"Neural Information Processing", Springer Science and Business Media... Crossref	<1%
157	Lu-ning Zhang, Jian-wei Liu, Zhi-yan Song, Xin Zuo. "Temporal attentio... Crossref	<1%
158	Monteiro, Andreia Alves Forte Oliveira. "Contributions to Spatial and Te... Publication	<1%
159	Universidade Estadual de Campinas on 2022-02-10 Submitted works	<1%
160	University College London on 2020-09-14 Submitted works	<1%
161	University of Cape Town on 2022-10-24 Submitted works	<1%
162	University of Warwick on 2019-09-13 Submitted works	<1%
163	Zhou, Zihao. "Neural Point Process for Learning Spatiotemporal Event ... Publication	<1%
164	Zilu Wen, Peihong Fu, Han Li, Chaoxiang Hu. "Sequence-to-schedule: A... Crossref	<1%